HOLBERTON SCHOOL DIJON

DOSSIER DE PROJET

CyberAudit & Solutions



Plateforme web de mise en relation entre PME/TPE et experts en cybersécurité pour la gestion d'audits de vulnérabilité

Titre professionnel visé :

Titre RNCP de Niveau 5 Développeur Web et Web Mobile

Candidat : Tommy JOUHANS

Rôle sur le projet : Développeur Front-End

Projet réalisé en binôme avec James ROUSSEL (Développeur Back-End)

Session du 16 au 27 Novembre 2026

Projet réalisé en formation : du 27 avril au 17 juillet 2026

Résumé

Pour donner suite à un projet d'entreprenariat réalisé avec des étudiant de différentes écoles à Dijon pour l'association des Entrep' Bourgogne ; CyberAudit & Solutions est une plateforme web full-stack destinée à connecter les petites et moyennes entreprises françaises dépourvues d'équipe informatique interne avec des experts en cybersécurité. Selon le Panorama de la cybermenace 2023 de l'ANSSI, les TPE, PME et ETI représentent 34 % des victimes de rançongiciels portées à la connaissance de l'agence, devant les collectivités territoriales (24 %), ces structures disposant rarement d'une équipe informatique dédiée capable de s'en prémunir.

Développée en binôme sur douze semaines selon une méthodologie agile, l'application permet à un client PME de soumettre une demande d'audit de sécurité en moins de trois minutes, de suivre son avancement à travers quatre statuts, puis de consulter un rapport de vulnérabilités vulgarisé au format PDF, généré automatiquement. Côté administrateur, un expert en cybersécurité gère l'ensemble des demandes et déclenche la génération des rapports.

Le projet repose sur une architecture découplée : React 19 / Tailwind CSS 4 côté client, API REST sécurisée en Django 5.1 / DRF, PostgreSQL 15 et Celery / Redis pour les tâches asynchrones. La sécurité a été traitée comme une exigence transversale : JWT avec rotation des jetons, RBAC, identifiants UUID anti-énumération, protection ORM contre les injections SQL, assainissement DOMPurify contre les attaques XSS et limitation du débit des requêtes.

Ce dossier présente le contexte et le cahier des charges du projet, sa gestion agile, ses spécifications fonctionnelles et techniques, puis les réalisations Front-End et Back-End argumentées, le jeu d'essai, la stratégie de tests (74 tests back-end, 64 tests front-End, couverture 91 %) et le déploiement continu sur Railway et Vercel.

Sommaire

Résumé 2
Sommaire 3
1. Introduction 5
1.1 Présentation de l'équipe et du contexte de formation 5
1.2 Présentation du projet et problématique 5
1.3 Persona et storytelling 5
2. Compétences du référentiel couvertes par le projet 6
3. Contexte du projet et expression des besoins 7
3.1 Liste des compétences mises en œuvre dans le cadre du projet 7
3.2 Expression des besoins 7
3.3 Contraintes du projet et livrables attendus 7
3.4 Environnement humain et technique 8
3.5 Objectifs de qualité 8
4. Gestion de projet 9
4.1 Méthodologie retenue 9
4.2 Planning et sprints 9
4.3 Outils de gestion et de communication 10
4.4 Suivi d'avancement, revues et rétrospectives 10
4.5 Gestion des risques 10
5. Spécifications fonctionnelles 11
5.1 Personas 11
5.2 User stories priorisées (méthode MoSCoW) 11
5.3 Maquettes et enchaînement des interfaces 11
6. Spécifications techniques 23
6.1 Architecture générale en couches 23
6.2 Stack technique et justification des choix 24
6.3 Flux de données : exemple de la soumission d'une demande d'audit 24
6.4 Accessibilité (RGAA / WCAG 2.1 niveau AA) 25
6.5 Budget de performance (Web Vitals) et éco-conception 25
7. Réalisations côté front-end 26
7.1 Installer et configurer son environnement de travail 26

7.2 Maquetter les interfaces utilisateur 27
7.3 Arborescence des composants React 28
7.4 Réaliser les interfaces utilisateur statiques 28
7.5 Développer la partie dynamique des interfaces utilisateur 29
7.6 Sécurité de la partie front-end 30
8. Réalisations côté back-end 32
8.1 Mettre en place une base de données relationnelle 32
8.2 Développer des composants d'accès aux données SQL et NoSQL 34
8.3 Développer des composants métier côté serveur 34
8.4 Sécurité de la partie back-end 35
8.5 Documenter le déploiement d'une application dynamique web ou web mobile 35
9. Jeu d'essai 37
10. Stratégie de tests et qualité 38
10.1 Tests back-end (pytest) 38
10.2 Tests front-end (Vitest et React Testing Library) 39
10.3 Tests de sécurité manuels 40
10.4 Tests d'accessibilité et de performance 40
11. Veille technologique et sécuritaire 41
11.1 Sources suivies 41
11.2 Méthode et traçabilité 41
11.3 Exemple concret de vulnérabilité suivie et traitée 41
12. Conclusion 42
12.1 Perspectives d'évolution 42
13. Remerciements 43
14. Annexes 44
Annexe 1 : Glossaire technique 44
Annexe 2 : Correspondance compétences REAC / sections du dossier 44
Annexe 3 : Matrice des risques complète 45
Annexe 4 : Bibliographie et sources 45
Annexe 5 : Diagrammes de conception complémentaires 46
Annexe 6 : Ressources complémentaires du projet 50
Annexe 7 : Top 10 des OSWAP 51

1. Introduction

1.1 Présentation de l'équipe et du contexte de formation

Ce projet a été conduit dans le cadre de ma formation au Titre RNCP de niveau 5 « Développeur Web et Web Mobile » à Holberton School Dijon, promotion 2026. Il s'agit d'un projet portfolio réalisé en formation, en binôme avec James ROUSSEL, sur une durée de douze semaines allant du 27 avril au 17 juillet 2026. La répartition des rôles a été clairement établie dès le lancement du projet : j'ai assuré l'intégralité du développement front-end (React, maquettage, interfaces, accessibilité, sécurité côté client), tandis que James ROUSSEL a pris en charge le développement back-end (Django, base de données, API REST, tâches asynchrones), la conception du modèle de données et les contrats d'interface ayant été établis conjointement.

Cette organisation en binôme, avec des périmètres techniques distincts mais une conception commune, reproduit les conditions réelles d'un projet de développement web en petite équipe, telles que décrites dans le référentiel emploi activités compétences (REAC) du titre : contributions croisées, revues de code obligatoires avant fusion, synchronisation des contrats d'interface entre les deux couches applicatives, et communication continue via des outils collaboratifs (Discord, GitHub, Notion, Trello).

1.2 Présentation du projet et problématique

Le point de départ du projet est un constat chiffré et documenté : selon le Panorama de la cybermenace 2023 de l'Agence nationale de la sécurité des systèmes d'information (ANSSI), les TPE, PME et ETI représentent 34 % des victimes de rançongiciels portées à la connaissance de l'agence en 2023, juste derrière les collectivités territoriales (24 %) ; un trio de tête identique depuis 2020, ces structures étant jugées « encore trop vulnérables ». Ces entreprises, souvent dépourvues d'équipe informatique interne, restent exposées à des cyberattaques de plus en plus fréquentes et sophistiquées : rançongiciels, hameçonnage ciblé, fuites de données personnelles. Les solutions de cybersécurité existantes sur le marché sont majoritairement conçues pour de grands groupes, avec des coûts et une complexité d'usage qui les rendent peu accessibles à un TPE ou une PME de quelques salariés.

CyberAudit & Solutions répond à cette problématique en proposant une plateforme web qui met en relation simple et directe une PME cliente avec un prestataire d'audit de cybersécurité. Le client soumet une demande d'audit en quelques clics parmi quatre formules (Audit, Sécurité, Protection, Premium), suit l'avancement de son dossier en temps réel à travers un tableau de bord dédié, puis reçoit un rapport de vulnérabilités rédigé en langage clair, accompagné d'un plan d'action sur trente jours. Le tout est délivré dans un environnement sécurisé, avec authentification et séparation stricte des rôles entre client et administrateur.

1.3 Persona et storytelling

Pour ancrer les choix de conception dans un besoin utilisateur concret, l'équipe a construit une persona principale, Marie, 45 ans, gérante d'un cabinet comptable de douze personnes à Dijon. Marie utilise Excel et Gmail au quotidien mais n'a aucune notion informatique avancée. Récemment ciblée par un courriel d'hameçonnage qui a failli compromettre les données de ses clients, elle cherche une solution simple et abordable pour faire auditer la sécurité de son cabinet, sans avoir à déchiffrer un rapport technique de deux cents pages. C'est cette persona qui a guidé l'ensemble des décisions d'ergonomie du projet : un parcours de soumission de demande d'audit réalisable en moins de trois minutes, un vocabulaire non technique dans l'interface, et un rapport de vulnérabilités vulgarisé plutôt que brut.

Une seconde persona, Karim, 32 ans, expert en cybersécurité, représente l'utilisateur côté administrateur : il gère la file des demandes reçues, réalise l'audit, saisit les résultats et déclenche la génération du rapport à destination du client.

2. Compétences du référentiel couvertes par le projet

Conformément au référentiel emploi activités compétences (REAC) du titre professionnel Développeur Web et Web Mobile, le projet CyberAudit & Solutions couvre l'intégralité des huit compétences professionnelles réparties sur les deux activités-types du titre. Le tableau ci-dessous liste ces compétences et renvoie vers les sections du présent dossier où chacune est mise en œuvre et argumentée.

Activité-type	Compétence professionnelle	Section
Développer la partie front-end d'une application web ou web mobile sécurisée	Installer et configurer son environnement de travail en fonction du projet web ou web mobile	7.1
	Maquetter des interfaces utilisateur web ou web mobile	7.2
	Réaliser des interfaces utilisateur statiques web ou web mobile	7.4
	Développer la partie dynamique des interfaces utilisateur web ou web mobile	7.5
Développer la partie back-end d'une application web ou web mobile sécurisée	Mettre en place une base de données relationnelle	8.1
	Développer des composants d'accès aux données SQL et NoSQL	8.2
	Développer des composants métier côté serveur	8.3
	Documenter le déploiement d'une application dynamique web ou web mobile	8.5

Les trois compétences transversales de l'emploi type communiquer en français et en anglais, mettre en œuvre une démarche de résolution de problème, et apprendre en continu sont illustrées tout au long du dossier : documentation technique rédigée en français et en anglais, gestion des blocages techniques rencontrés en sprint (section 4.4), et veille technologique et sécuritaire continue (section 11).

3. Contexte du projet et expression des besoins

Ce projet ayant été réalisé en formation et non au sein d'une entreprise d'accueil, cette section présente, conformément aux exigences du référentiel, la liste des compétences mises en œuvre, l'expression des besoins ayant permis de définir les objectifs et les limites du projet, l'environnement technique retenu, ainsi que les réalisations attendues.

3.1 Liste des compétences mises en œuvre dans le cadre du projet

Le projet a permis de mettre en œuvre l'ensemble des compétences professionnelles des deux activités-types du titre Développeur Web et Web Mobile, détaillées dans le tableau de la section 2, ainsi que les trois compétences transversales de communication, de résolution de problème et d'apprentissage continu.

3.2 Expression des besoins

L'expression des besoins a été formalisée collectivement par l'équipe à partir du constat ANSSI présenté en introduction et affinée à travers la construction du persona Marie. Les besoins fonctionnels retenus sont les suivants :

- Permettre à un client PME de créer un compte et de s'authentifier de façon sécurisée.
- Lui permettre de soumettre une demande d'audit de sécurité en choisissant l'une des quatre formules proposées, en moins de trois minutes.
- Lui permettre de suivre l'état d'avancement de sa demande (en attente, en cours, terminée, archivée).
- Le notifier automatiquement par courriel à chaque changement de statut.
- Lui donner accès, une fois l'audit terminé, à un rapport de vulnérabilités au format PDF, rédigé en langage clair et accompagné d'un plan d'action.
- Permettre à un administrateur expert en cybersécurité de consulter l'ensemble des demandes, de modifier leur statut et de déclencher la génération du rapport.
- Mettre à disposition des clients des modules de sensibilisation à la cybersécurité (hameçonnage, authentification, sauvegarde des données).
- Garantir la sécurité et la confidentialité des données traitées, à chaque étape du parcours utilisateur, et le consentement RGPD du client à l'inscription.

Ces besoins ont ensuite été priorisés collectivement selon la méthode MoSCoW (Must have, Should have, Could have, Won't have), détaillée en section 5.2, afin de définir un périmètre de produit minimum viable réalisable sur douze semaines.

3.3 Contraintes du projet et livrables attendus

Le projet est soumis à une contrainte de délai ferme : douze semaines, calendrier imposé par la formation, avec des jalons hebdomadaires (planning détaillé en section 4.2). Il est également soumis à des contraintes techniques : l'application doit être intégralement sécurisée dès la conception (approche « security by design »), accessible aux personnes en situation de handicap (conformité RGAA visée), et déployée en production avec HTTPS avant la soutenance finale.

Les livrables attendus en fin de projet sont les suivants :

- Une application web fonctionnelle, déployée et accessible publiquement (front-end sur Vercel, back-end sur Railway).
- Le code source versionné sur GitHub, avec historique de commits conventionnels et pull requests documentées.
- Une suite de tests automatisés (unitaires et d'intégration) côté front-end et back-end.

- Une chaîne d'intégration et de déploiement continus (CI/CD) opérationnelle.
- Une documentation technique complète (architecture, base de données, API, sécurité, déploiement).
- Le présent dossier de projet et son support de présentation pour la certification RNCP.

3.4 Environnement humain et technique

Environnement humain

Rôle	Personne	Responsabilité
Développeur Front-End	Tommy JOUHANS	Maquettage, interfaces utilisateur, accessibilité, sécurité côté client, déploiement Vercel
Développeur Back-End	James ROUSSEL	Modélisation des données, API REST, sécurité côté serveur, déploiement Railway
Référent technique	Formateur Holberton	Accompagnement pédagogique et validation technique hebdomadaire
Relecteur de projet	Formateur Holberton	Relecture et retours sur la documentation et le dossier de projet

Environnement technique

L'environnement de travail retenu s'appuie sur les outils et technologies suivants, détaillés et justifiés en section 6.2 :

- Poste de développement individuel sous environnement de développement intégré (VS Code), avec extensions ESLint, Python et Git.
- Gestion de versions Git, dépôt distant hébergé sur GitHub (dépôt « portfolio »).
- Environnement de développement local : Node.js, Python, PostgreSQL, Redis.
- Outils de conteneurisation Docker pour l'environnement back-end et sa base de données de test.
- Hébergement de production : Vercel pour le front-End (CDN, HTTPS automatique), Railway pour le back-end, la base de données PostgreSQL et le broker Redis.
- Outils collaboratifs : Discord pour la communication quotidienne, Notion pour la documentation, Trello pour le suivi des tâches, GitHub Actions pour l'intégration continue.

3.5 Objectifs de qualité

Quatre objectifs SMART (Spécifique, Mesurable, Atteignable, Réaliste, Temporellement défini) ont structuré la démarche qualité du projet dès son lancement.

Objectif	Détail SMART	Résultat
Authentification et rôles	Système JWT avec deux rôles distincts protégeant toutes les routes sensibles ; 100 % des routes testées via Postman ; livré fin de semaine 3	Atteint
Gestion des demandes d'audit	Cycle CRUD complet avec quatre statuts ; cinq scénarios de test validés ; démontrable fin de semaine 4	Atteint
Déploiement et sécurité en production	Déploiement HTTPS avec protections anti-XSS et anti-injection SQL validées ; URL publique accessible avant la soutenance	Atteint
Expérience utilisateur	Un utilisateur non technicien (persona Marie) soumet une demande sans aide, en moins de trois minutes, validé par trois tests utilisateurs	Atteint (S7)

4. Gestion de projet

4.1 Méthodologie retenue

Le projet a été conduit selon une démarche agile inspirée de Scrum, adaptée à une équipe de deux développeurs : douze semaines de développement organisées en sprints hebdomadaires ou bi-hebdomadaires, chacun clôturé par une démonstration (Sprint Review) et une rétrospective. Cette approche présente l'avantage de livrer des incréments fonctionnels testables à intervalles réguliers, de détecter tôt les écarts entre le planning et la réalité, et de réajuster les priorités à chaque itération. Elle impose en contrepartie une discipline de synchronisation quotidienne entre les deux développeurs, en particulier pour définir les contrats d'interface entre le Front-End et le Back-End avant de coder de part et d'autre.

4.2 Planning et sprints

Sprint	Période	Objectif principal	Statut
S1	27 avril	Conception et maquettes (Figma), user stories	Terminé
S2	4 mai	Mise en place de l'environnement (React/Vite, Django/PostgreSQL)	Terminé
S3	11-17 mai	Authentification et sécurité JWT	Terminé
S4	18-24 mai	Fonctionnalités cœur 1 : formulaire de demande, tableau de bord client	Terminé
S5	25 mai-1 juin	Fonctionnalités cœur 2 : génération PDF, tableau de bord admin, formations	Terminé
S6	1-7 juin	Tâches asynchrones (Celery), notifications, CI/CD	Terminé
S7-S8	8-21 juin	Tests approfondis, 62 tests Front-End Vitest, corrections de bugs	Terminé
S9-S10	22 juin-5 juillet	Déploiement Railway (Back-End) et Vercel (Front-End)	Déployé
S11-S12	6-17 juillet	Stabilisation et préparation de la soutenance finale du Demoday	Terminé

Sur les quarante points de complexité planifiés sur l'ensemble du projet, trente-huit ont été livrés, soit une vélocité de 95 %. Les deux points non livrés dans les délais initiaux (tests Cypress de bout en bout) ont été volontairement reportés en fin de projet, la priorité ayant été donnée à la couverture de tests unitaires et d'intégration, jugée plus critique pour la certification.

4.3 Outils de gestion et de communication

Outil	Usage	Fréquence
GitHub	Versionnement du code, pull requests obligatoires, revues croisées	À chaque commit
GitHub Actions	Intégration continue : lint, tests, build à chaque push	Automatique
Discord	Communication quotidienne (canaux frontend, backend, général)	Quotidienne
Notion	Documentation technique et suivi des tâches	Après chaque session
Trello	Tableau kanban de suivi des tâches par sprint	Quotidienne
Point hebdomadaire	Bilan de sprint et replanification	Chaque semaine

4.4 Suivi d'avancement, revues et rétrospectives

Chaque sprint s'est conclu par une démonstration des fonctionnalités livrées et une rétrospective structurée autour de trois questions : ce qui a bien fonctionné, les difficultés rencontrées, et les actions à mettre en œuvre pour le sprint suivant. À titre d'exemple, la rétrospective du sprint 2 (tableau de bord et demandes d'audit) a permis d'identifier une difficulté récurrente de synchronisation des interfaces JSON entre le front-end et l'API réelle ; « l'action corrective retenue a été de définir systématiquement les contrats d'interface en amont du développement, dès le sprint suivant, ce qui a nettement fluidifié la suite du projet ».

Le format de point quotidien adopté, inspiré du Daily Stand-up Scrum, a été formalisé par un message asynchrone sur Discord selon la structure : tâches terminées la veille, tâches en cours, blocages éventuels. Cette discipline a permis de détecter rapidement les dépendances bloquantes, par exemple lorsque le développement du tableau de bord administrateur a dû attendre la finalisation du contrôle d'accès par rôle côté back-end.

4.5 Gestion des risques

Risque identifié	Probabilité	Impact	Mesure de mitigation
Retard sur l'intégration front/back par divergence des contrats d'API	Moyenne	Élevé	Définition des schémas JSON en amont (S1), localStorage côté front en attendant l'API réelle
Faille de sécurité non détectée avant la mise en production	Moyenne	Élevé	Tests de sécurité manuels dédiés (S7-S10), revue de code croisée systématique, veille OWASP
Non-respect du délai de douze semaines	Faible	Élevé	Découpage en sprints avec priorisation MoSCoW, points hebdomadaires de replanification
Indisponibilité d'un des deux développeurs	Faible	Moyen	Documentation continue sur Notion, séparation claire des périmètres front/back
Dépassement du budget de performance (Web Vitals)	Faible	Moyen	Lighthouse CI intégré au pipeline, budget de performance défini dès la conception (section 6.5)

5. Spécifications fonctionnelles

Les spécifications fonctionnelles décrivent le périmètre fonctionnel du projet et les interactions attendues avec l'utilisateur, en amont des phases de maquettage et de développement. Elles se distinguent des spécifications techniques (section 6), qui traduisent ces besoins en solutions d'architecture, de technologies et de sécurité.

5.1 Personas

Deux personas ont guidé l'ensemble des choix fonctionnels du projet : Marie, 45 ans, gérante d'un cabinet comptable sans compétence informatique, représentant l'utilisateur client ; et Karim, 32 ans, expert en cybersécurité, représentant l'utilisateur administrateur qui traite les demandes d'audit en parallèle de son activité de conseil.

5.2 User stories priorisées (méthode MoSCoW)

Les besoins fonctionnels ont été formulés sous forme d'user stories et priorisés selon la méthode MoSCoW pour définir le périmètre du produit minimum viable réalisable sur douze semaines.

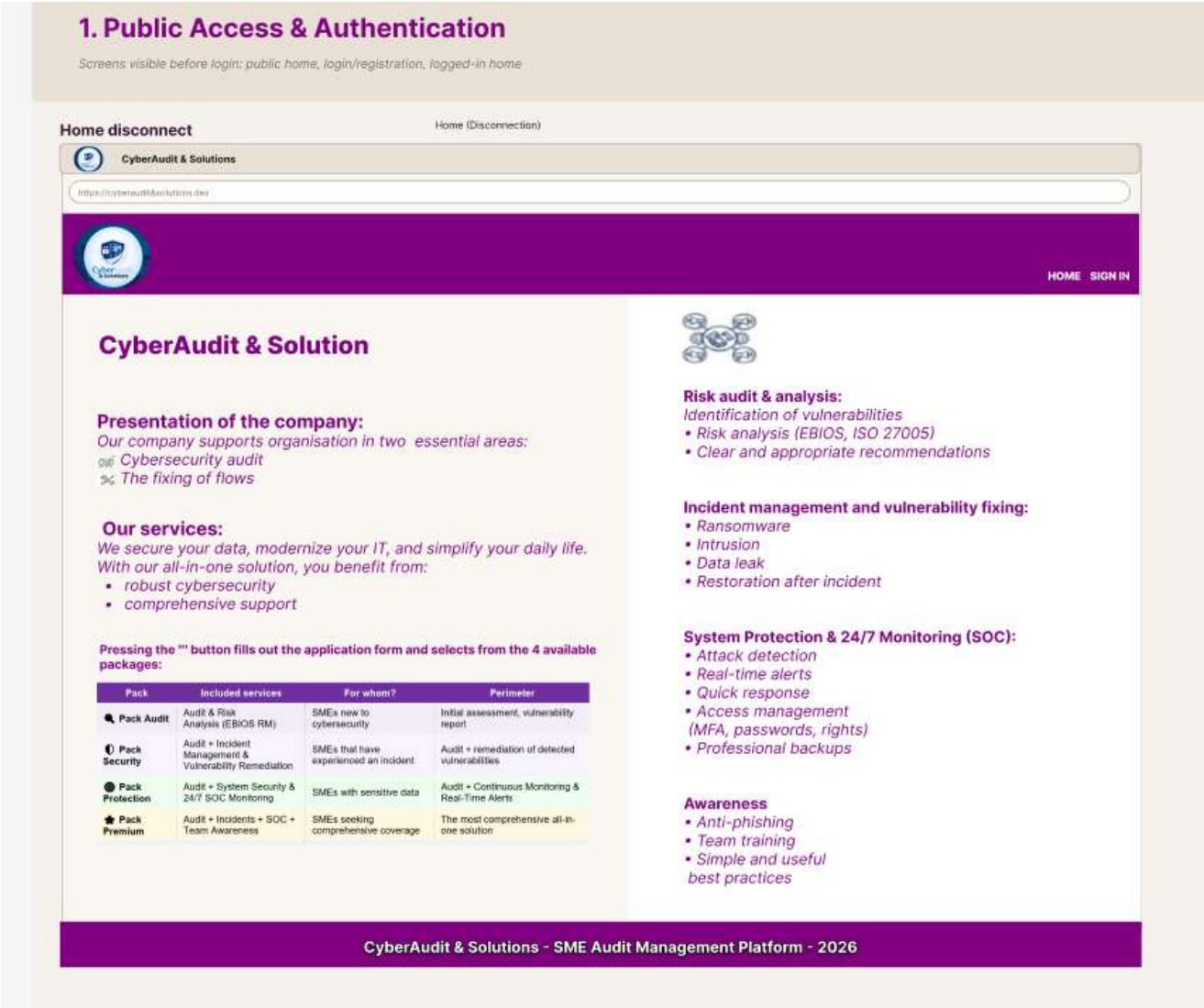
Must have (fonctionnalités indispensables)

Identifiant	User story	Sprint
US-01	En tant que visiteur, je veux créer un compte afin d'accéder à la plateforme	S3
US-02	En tant qu'utilisateur, je veux me connecter afin d'accéder à mon espace	S3
US-03	En tant qu'utilisateur, je veux me déconnecter afin de protéger mon compte	S3
US-04	En tant que client PME, je veux soumettre une demande en moins de 3 minutes	S4
US-05	En tant que client, je veux suivre le statut de ma demande (4 statuts)	S4
US-06	En tant que client, je veux recevoir un courriel à chaque changement de statut	S4
US-07	En tant qu'administrateur, je veux voir toutes les demandes triées	S5
US-08	En tant qu'administrateur, je veux modifier le statut d'une demande	S5
US-09	En tant qu'administrateur, je veux générer un rapport PDF	S5
US-10	En tant que client, je veux télécharger mon rapport PDF	S5

Les user stories de priorité Should Have ont porté sur les modules de sensibilisation, la réinitialisation de mot de passe, les filtres administrateur et les notifications visuelles dans l'interface. Les priorités Could have et Won't have (score de sécurité par intelligence artificielle, application mobile native, marketplace de consultants, authentification OAuth2) ont été explicitement exclues du périmètre de la version 1 et documentées comme perspectives d'évolution (section 12.1).

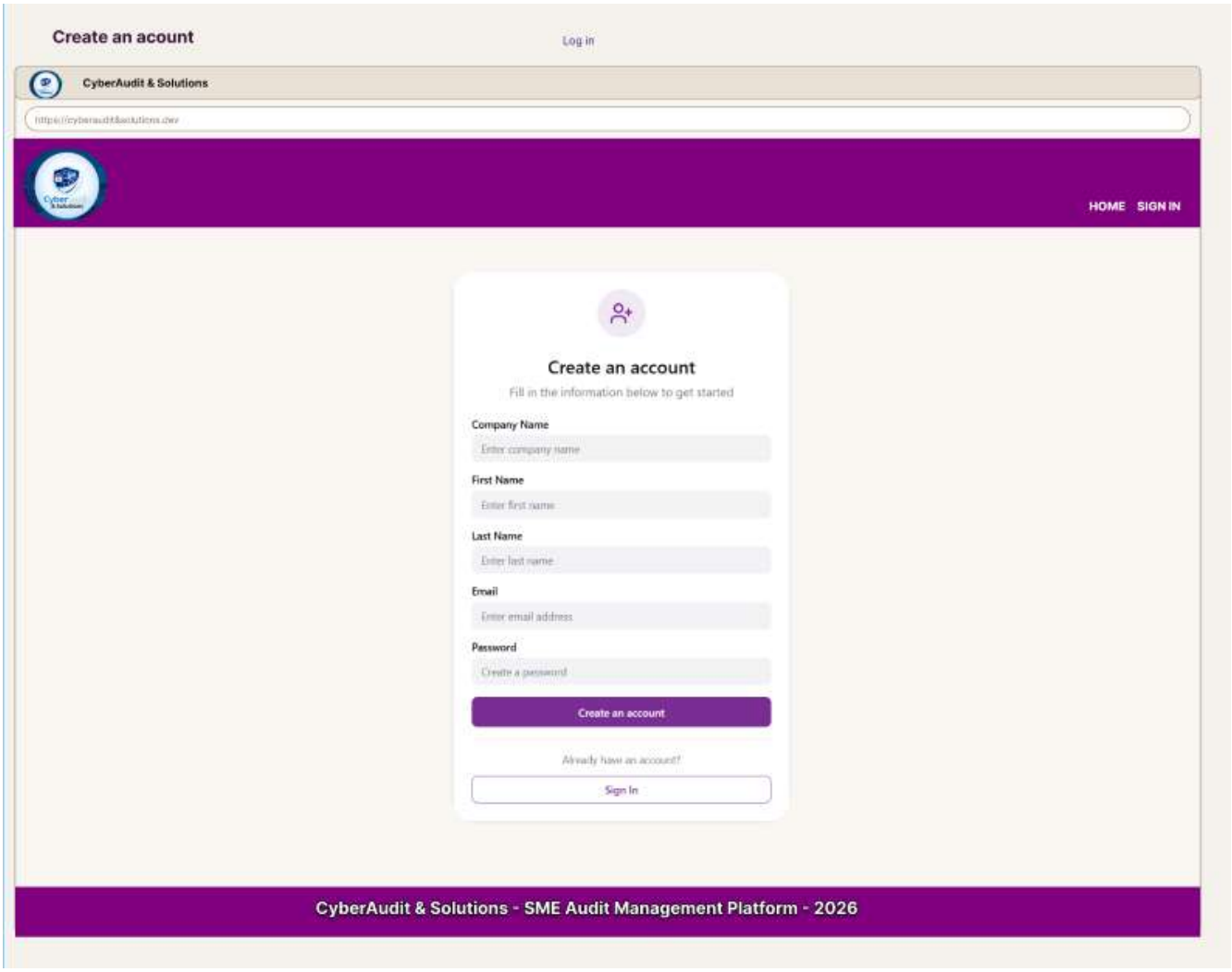
5.3 Maquettes et enchaînement des interfaces

Onze écrans ont été maquettés sur Figma dès le premier sprint, avant tout développement, conformément à la démarche de conception centrée utilisateur. Chaque écran a été conçu en version desktop et en version mobile (Breakpoints Tailwind sm/md/lg, approche « mobile first »), conformément à l'exigence d'adaptation web et web mobile du référentiel ; les captures ci-dessous présentent la version desktop des maquettes principales, la version mobile étant vérifiable sur le fichier Figma du projet et sur la démonstration en ligne. Chaque maquette a été pensée en priorité pour la persona Marie, avec un objectif constant de simplicité et de minimalité des affichages.



Écran 1 : page d'accueil publique « Présentation de l'entreprise, des services et des formules, avec appel à l'action vers l'inscription ».

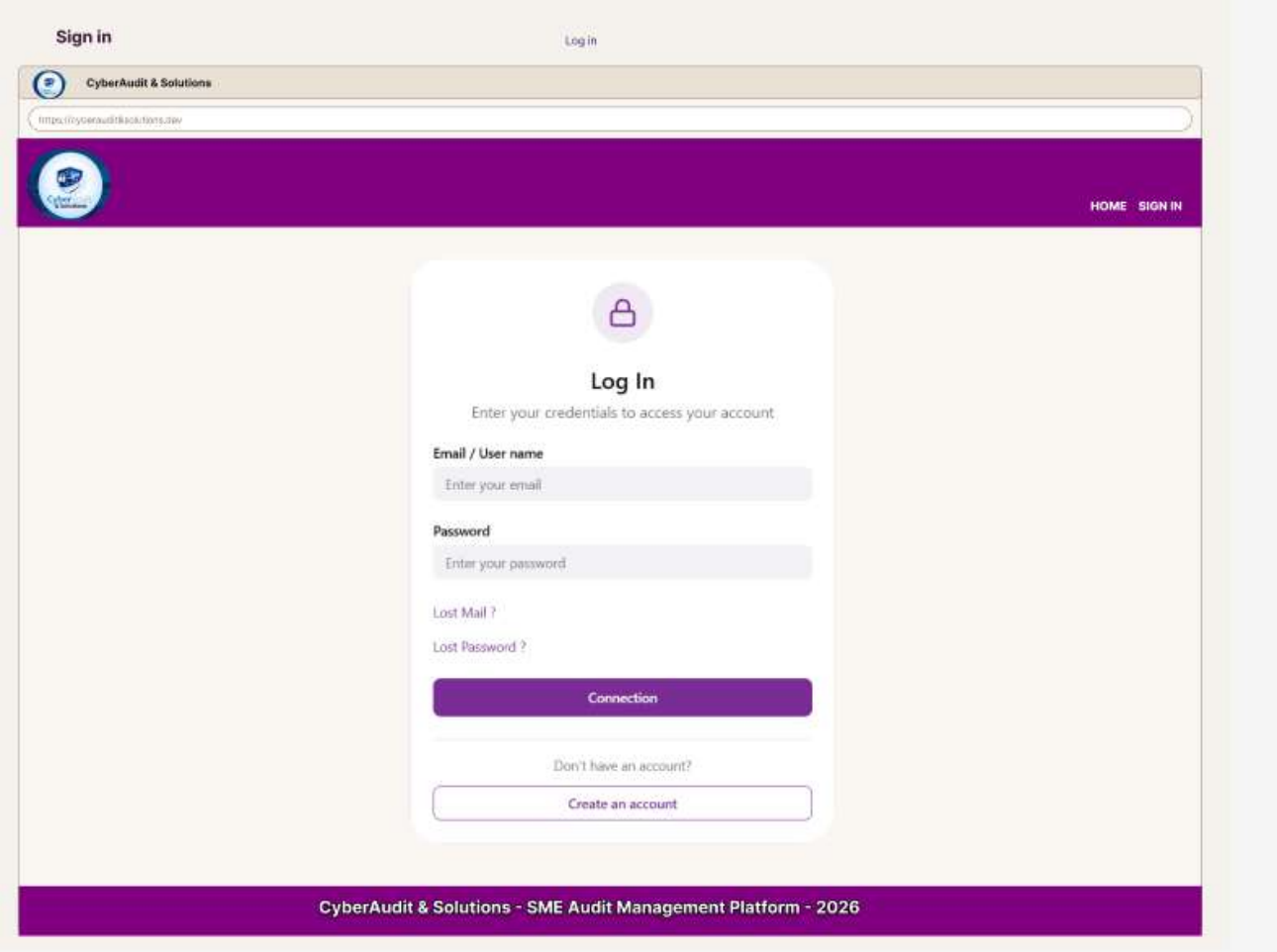
La page d'accueil de la plateforme CyberAudit & Solutions présente l'entreprise, ses services et les packs proposés, taillés pour les entrepreneurs de moins de 50 collaborateurs. Il suffit de se rendre dans l'onglet « Sign in » pour que la cliente (Marie) puisse créer un compte et se connecter.



Écran 2 : Inscription avec un courriel, un mot de passe, un consentement RGPD/CGU. Règles de validation détaillées en section 7.4.

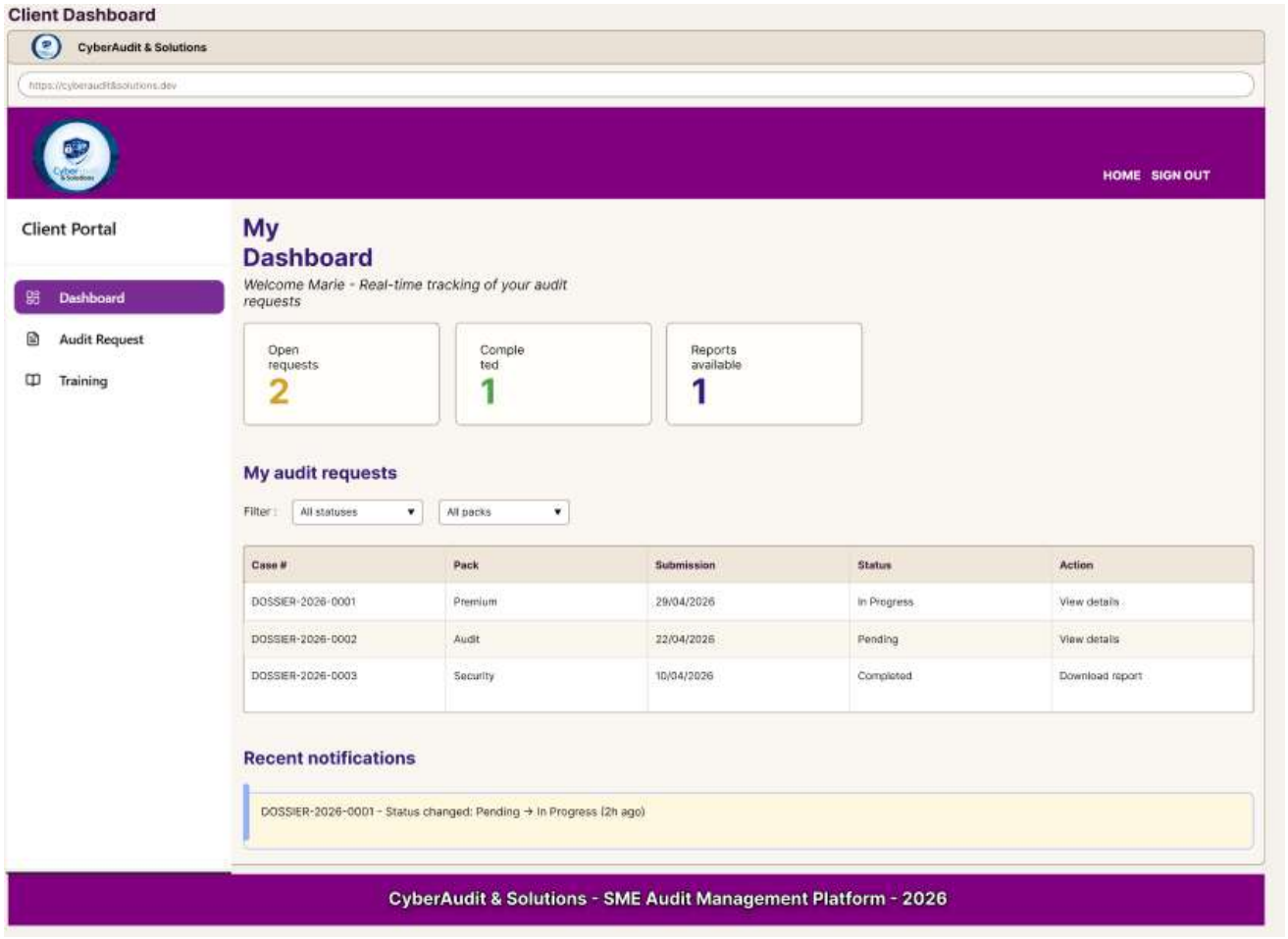
Après avoir cliqué sur « Créer un compte », la cliente saisit le nom de son entreprise, son prénom, son nom, son courriel et son mot de passe (haché côté serveur), et coche une case de consentement RGPD/CGU obligatoire à la soumission du formulaire.

Règles de remplissage du formulaire : chaque champ contient au maximum 50 caractères ; si un champ est manquant, le message d'erreur « Veuillez renseigner les informations du formulaire » s'affiche ; le format courriel (nom@domaine) est requis ; le mot de passe doit comporter une minuscule, une majuscule, un chiffre, un caractère spécial et dix caractères minimum.



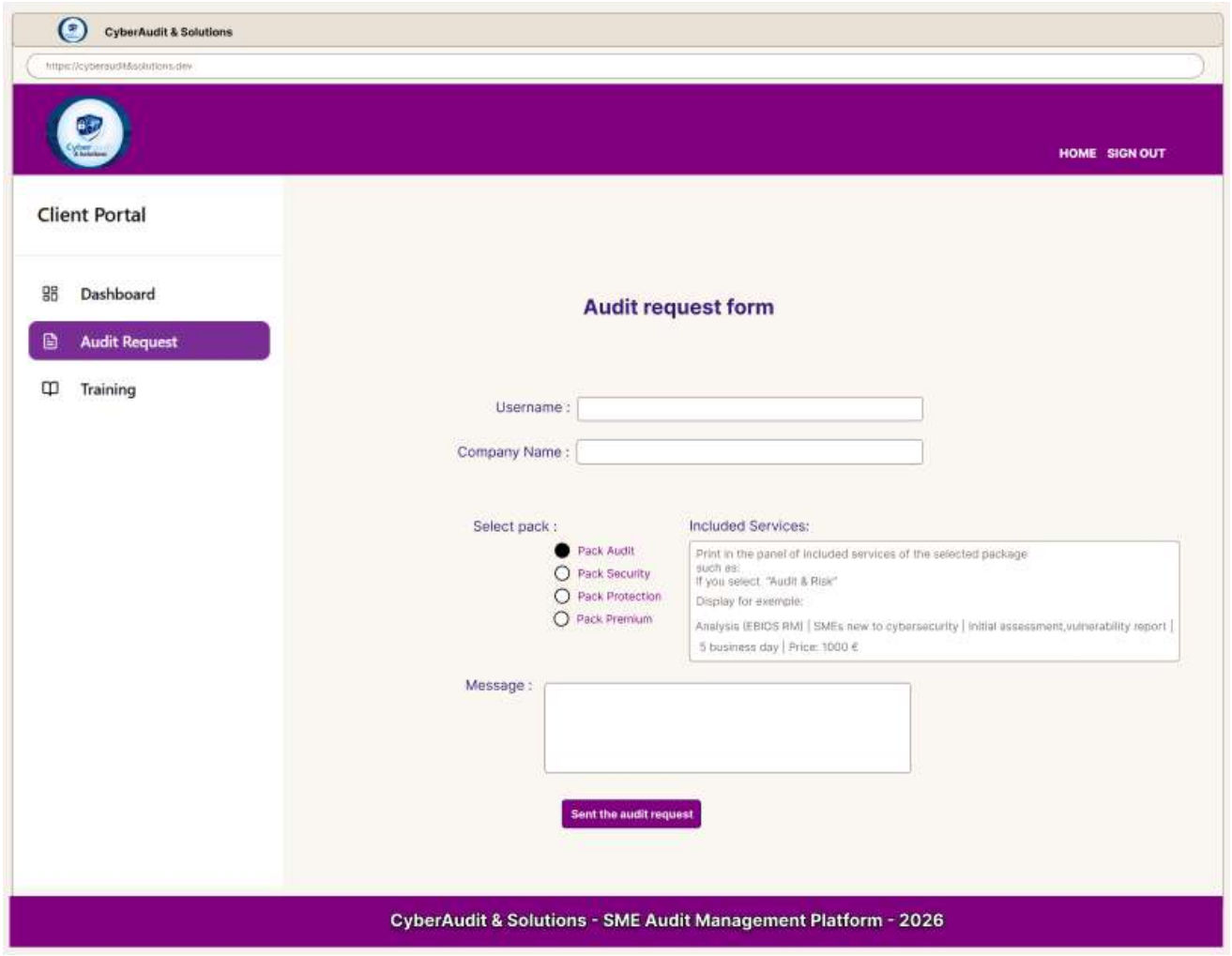
Écran 3 : Connexion par courriel et mot de passe, lien vers la création de compte.

La cliente peut se connecter avec son compte ; sinon, elle clique sur « Créer un compte ». Les liens « Courriel oublié » et « Mot de passe oublié » sont prévus dans la maquette (fonctionnalité de priorité Should have, à développer ultérieurement).



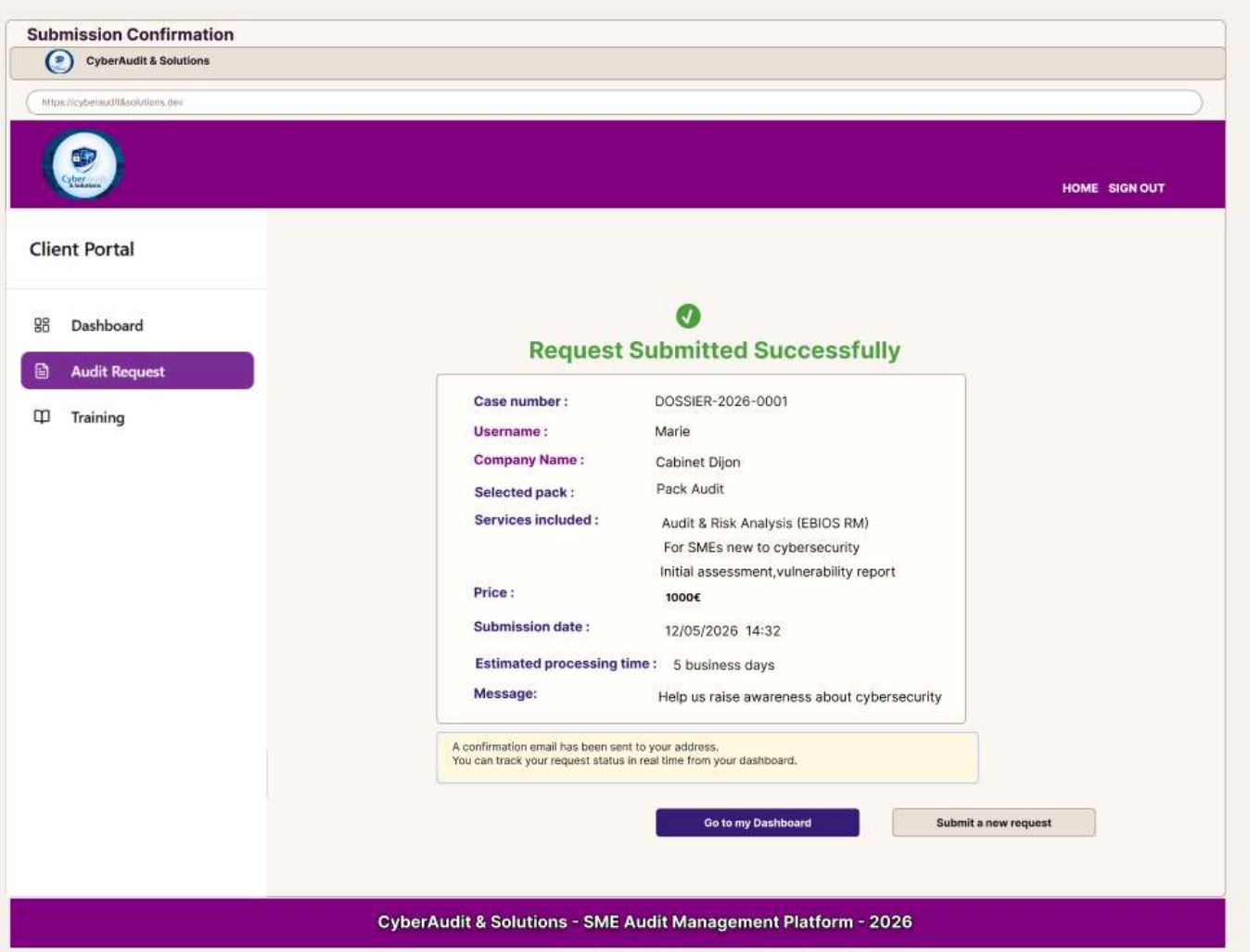
Écran 4 : Tableau de bord client « Liste des demandes avec statut coloré, fil de notifications et action contextuelle par ligne (voir le détail ou télécharger le rapport disponible) ».

Une fois connectée, la cliente consulte son tableau de bord pour suivre l'avancement de chacune de ses demandes d'audit (En attente / En cours / Terminée / Archivée), affichées sous forme de liste avec un badge de statut coloré et un fil de notifications récentes (par exemple un changement de statut). La colonne « Action » propose « Voir les détails » pour une demande en cours (écran 6) ou « Télécharger le rapport » lorsque le statut est Terminé (écran 11, il s'agit de la même interface, la fonctionnalité affichée dépendant du statut de la demande). L'onglet « Audit Request » permet de créer une nouvelle demande (écran 5).



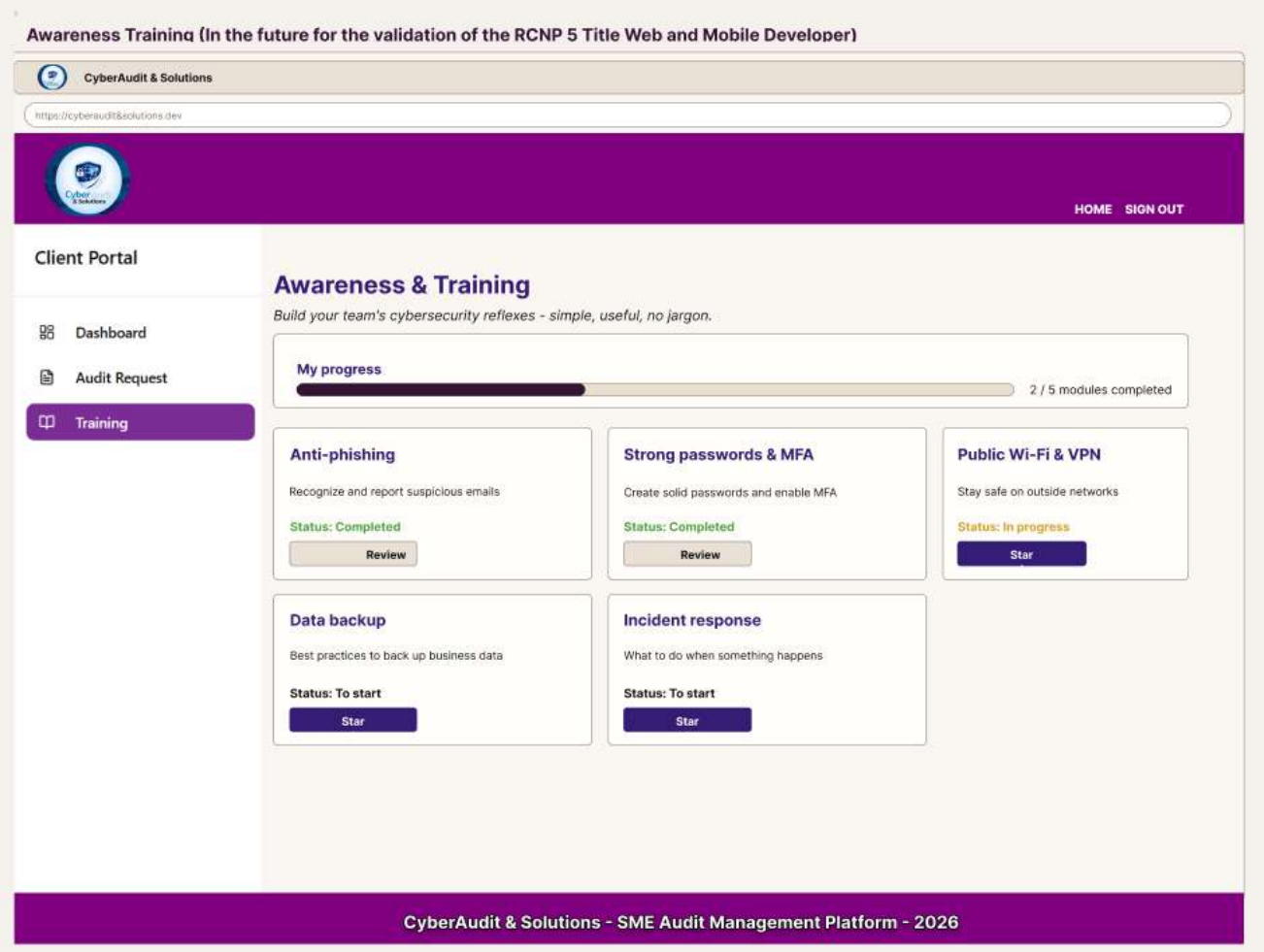
Écran 5 : Formulaire de demande d'audit « Nom d'utilisateur, nom de l'entreprise, choix de la formule et message. Objectif : moins de trois minutes ».

En cliquant sur l'onglet « Audit Request », la cliente accède au formulaire. Elle doit renseigner : son nom d'utilisateur (50 caractères maximum) et le nom de l'entreprise (moins de 50 caractères) ; choisir le pack (Audit / Sécurité / Protection / Premium) ; la sélection d'un pack via le bouton radio affiche ses détails dans le panneau « Included Services » (public visé, périmètre d'analyse, estimation de la durée et tarif) ; et rédiger un message optionnel décrivant la demande souhaitée. Le formulaire se soumet via « Sent the audit request ».



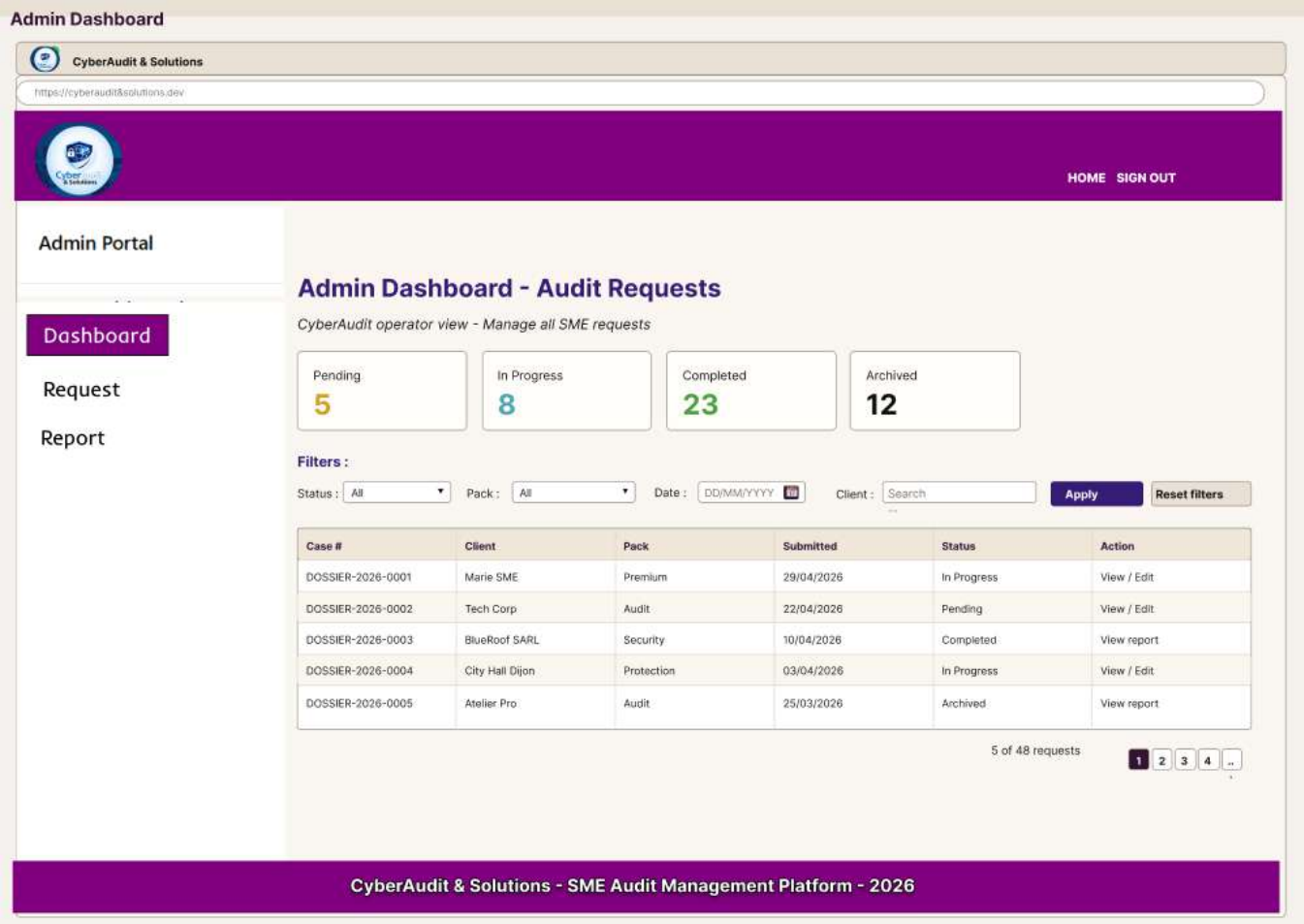
Écran 6 : Confirmation d'envoi d'une demande côté client, récapitulatif de la demande et accusé de réception.

Après l'envoi de la demande, un écran de confirmation reprend le récapitulatif de la demande (numéro de dossier, formule choisie, services inclus, prix, délai estimé et message transmis), et confirme qu'un courriel de confirmation a été envoyé à la cliente par une tâche asynchrone Celery qui appelle l'API du service EmailJS. Le suivi ultérieur du statut (changements successifs, consultation du détail) s'effectue ensuite depuis le tableau de bord et son fil de notifications (écran 4).



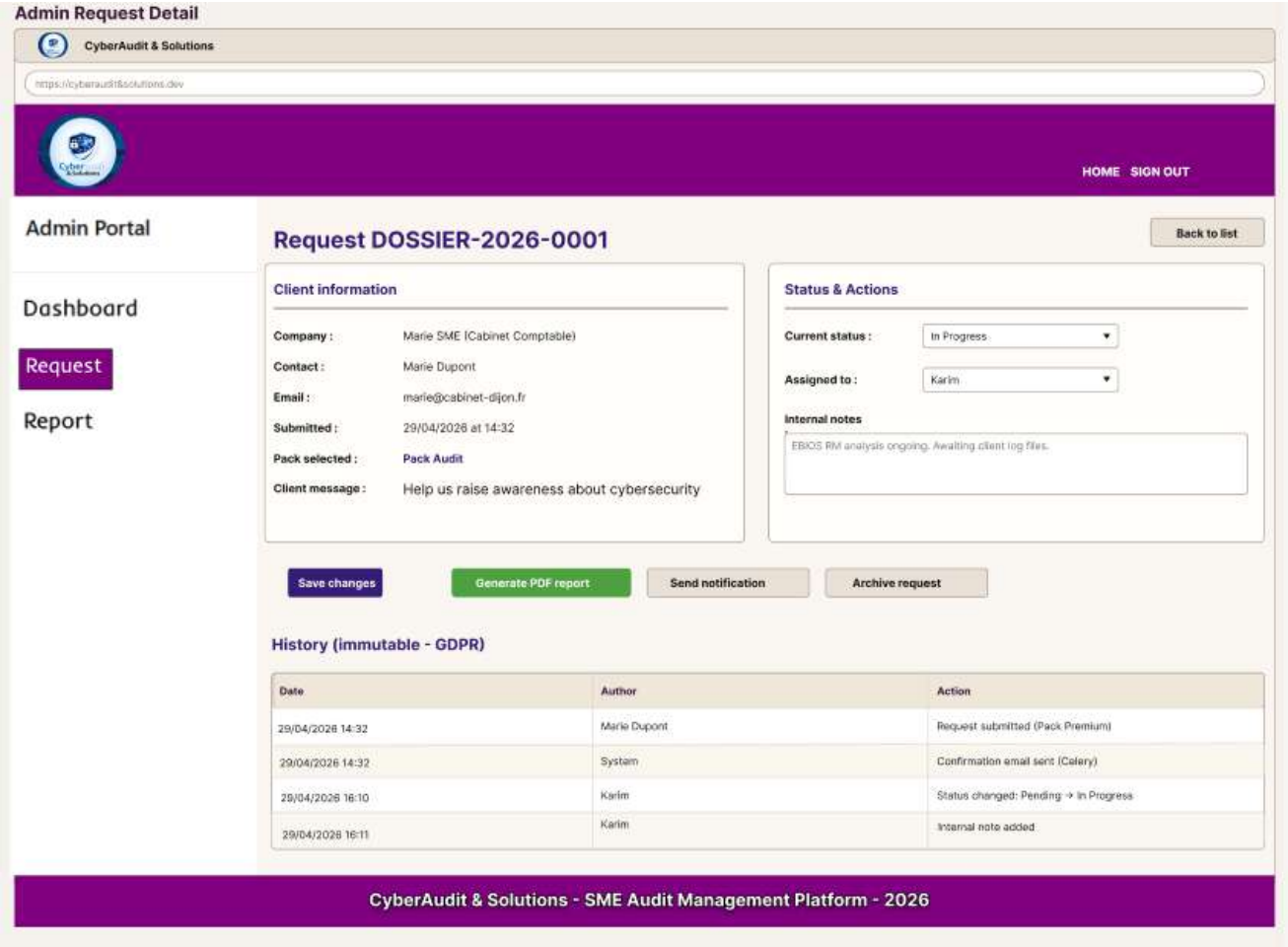
Écran 7 : Modules de sensibilisation à la cybersécurité (hameçonnage, authentification multifacteurs, sauvegarde des données).

Cinq modules de sensibilisation sont proposés : anti-hameçonnage, authentification multifacteurs, sécurité des connexions publiques et Wi-Fi/VPN, sauvegarde des données, réponse aux incidents, avec une barre de progression par utilisateur (ici deux modules sur cinq déjà validés). Ils sont prévus sous différents formats (vidéo, visioconférence, documentation). Cette fonctionnalité, de priorité Should have, a été maquettée pendant la formation ; son développement applicatif complet dépasse le périmètre couvert par le présent titre et sera poursuivi dans une version ultérieure du produit (section 12.1), une gamification des modules avec des cours et vidéo de sensibilisation seront déployé prochainement au-delà du passage du titre RNCP.



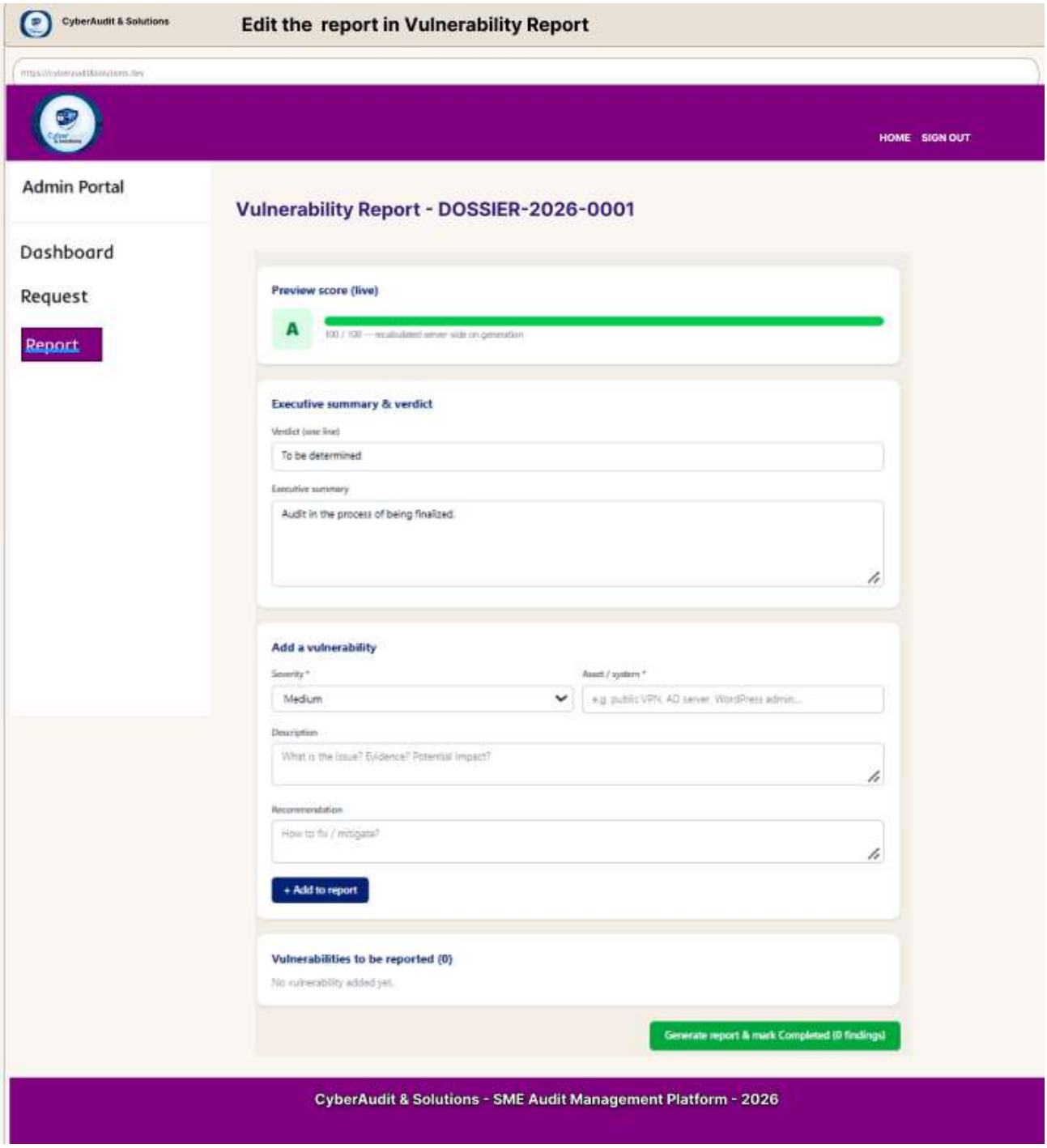
Écran 8 : tableau de bord administrateur en vue globale des demandes avec compteurs par statut et filtres par statut, par formule, par date et par client.

L'expert (administrateur) accède à un tableau de bord rassemblant toutes les demandes reçues, avec des compteurs synthétiques (en attente, en cours, terminées, archivées). Il peut filtrer la liste selon le statut, le pack, la date ou le client, puis consulter et modifier chaque demande en cliquant sur « View / Edit ».



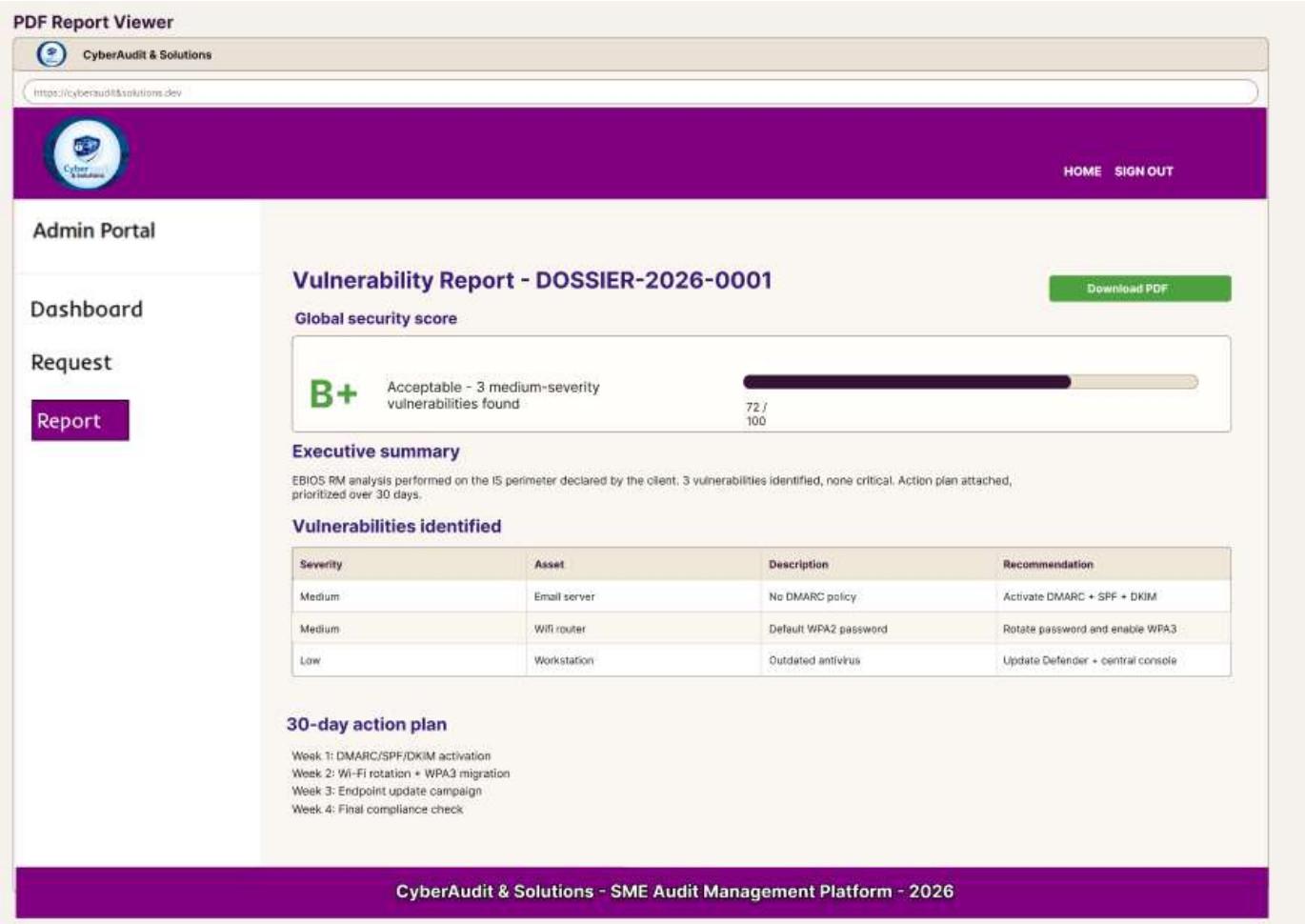
Écran 9 : Détail d'une demande côté administrateur « changement de statut, attribution, notes internes, génération du rapport PDF, envoi de notification, historique immuable ».

Depuis la vue détaillée, l'expert consulte les informations de la cliente, change le statut et l'attribution via le panneau « Status & Actions », ajoute des notes internes, génère un rapport PDF via « Generate PDF report », envoie une notification à la cliente et archive la demande. Un historique immuable (exigence RGPD) trace chaque action horodatée : soumission, envoi de l'accusé Celery, changement de statut, ajout de note.



Écran 10 : Le rapport de vulnérabilité à éditer l'administrateur

L'administrateur peut rédiger un rapport de vulnérabilité en fonction du niveau de la gravité, de l'atout concerné, la description et des recommandations pour le suivi de demande d'audit au client. Une fois que le rapport est rédigé, l'administrateur peut le générer en PDF.



Écran 11 : Aperçu du rapport de vulnérabilités « score de sécurité global, actifs analysés, description, recommandations, plan d'action à 30 jours ».

Le rapport affiche un score de sécurité global sur 100 avec une note synthétique (par exemple B+ / Acceptable – 3 medium-severity), un résumé exécutif, les vulnérabilités identifiées (niveau de gravité, actif concerné, description et recommandation) et un plan d'action à 30 jours généré automatiquement, téléchargeable au format PDF.

Schéma d'enchaînement des maquettes

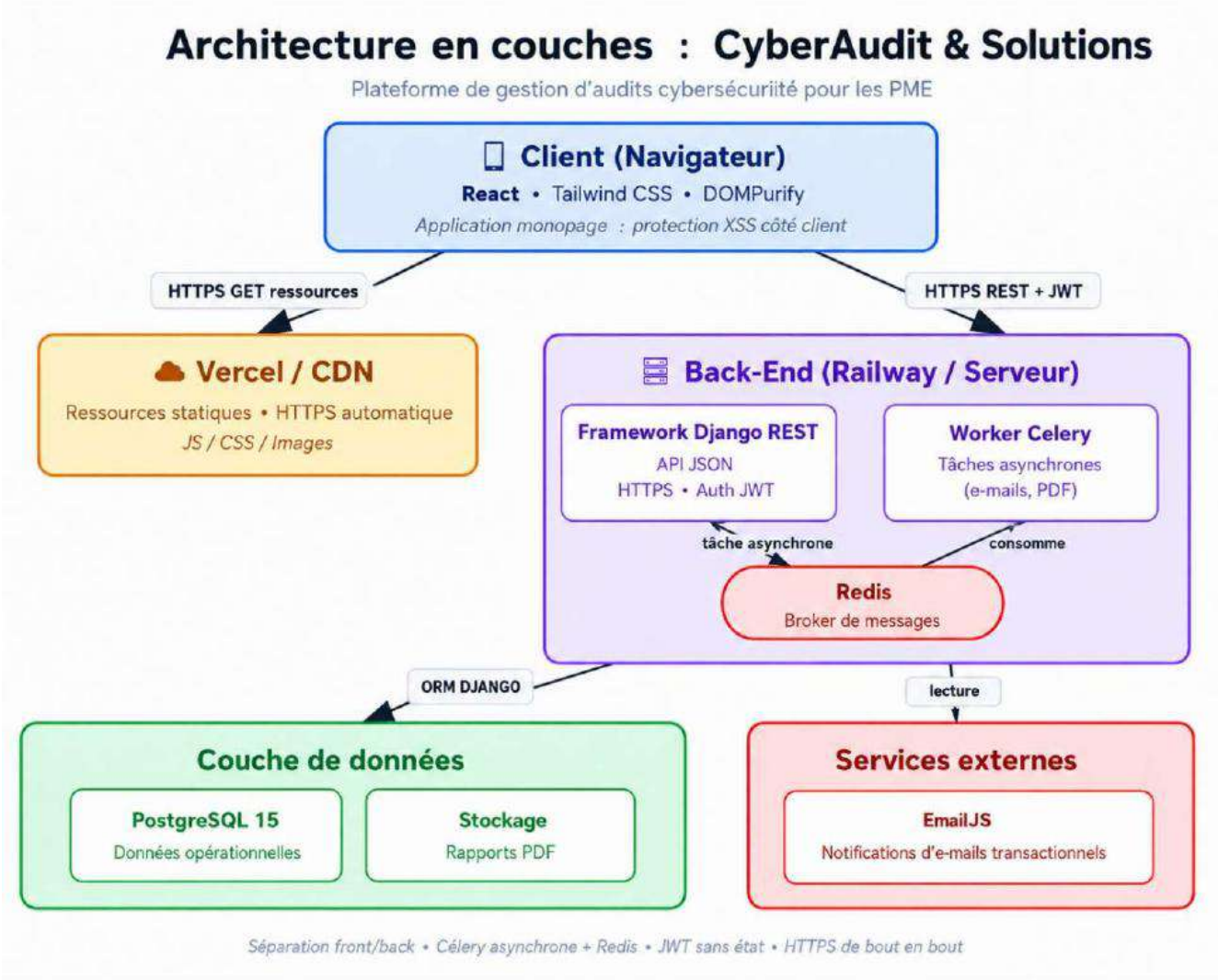
L'enchaînement des écrans suit un parcours linéaire pour le client : de la page d'accueil publique (écran 1), l'utilisateur non authentifié est dirigé vers l'inscription (écran 2) ou la connexion (écran 3). Une fois authentifié, il accède à son tableau de bord (écran 4), point d'entrée vers la création d'une nouvelle demande (écran 5), la consultation de la confirmation d'envoi (écran 6), les modules de formation (écran 7) ou le téléchargement d'un rapport disponible (écran 11, depuis le même tableau de bord une fois le rapport généré en écran 10). Côté administrateur, l'enchaînement bascule après authentification vers le tableau de bord global (écran 8), puis vers le détail et le traitement d'une demande individuelle (écran 9), qui déclenche la génération du rapport consulté en écran 10. Ce schéma de navigation a été formalisé et validé collectivement avant le début du développement des interfaces statiques (section 7.4) ; sa version graphique (arborescence des composants et logique de routage) figure en Annexe 5.

6. Spécifications techniques

Les spécifications techniques traduisent les besoins fonctionnels en solutions concrètes d'architecture, de technologies et de sécurité. Elles couvrent l'environnement, le matériel, l'hébergement, l'accessibilité et la maintenance de l'application.

6.1 Architecture générale en couches

L'application repose sur une architecture en couches découplée, qui sépare clairement la présentation (client React), la logique métier et l'accès aux données (API Django) et la persistance (PostgreSQL, Redis). Ce découplage front/back permet deux déploiements indépendants, un contrôle fin des origines autorisées (CORS), et la mise en cache de l'application front-end statique sur un réseau de diffusion de contenu (CDN).



Architecture en couches de CyberAudit & Solutions « client React/Vite, API Django/DRF, worker asynchrone Celery, bases de données PostgreSQL et Redis ».

6.2 Stack technique et justification des choix

Couche	Technologie	Justification
Front-end	React + Vite + Tailwind CSS	SPA performante, système de conception rapide à mettre en œuvre, écosystème mature
Back-end	Django + Django REST Framework	Sécurité native (ORM, protection CSRF/XSS), écosystème mature, productivité élevée
Base de données	PostgreSQL	SGBD relationnel ACID, index performants, gestion fine des types (UUID, JSON)
Tâches asynchrones	Celery + Redis	Génération de PDF hors du thread HTTP (2 à 5 secondes), file de messages persistante
Authentification	SimpleJWT	Authentification sans état (stateless), scalable horizontalement
Génération PDF	WeasyPrint	Conversion HTML/CSS vers PDF, réutilisation du système de conception web
Hébergement back-end	Railway	HTTPS automatique, déploiement continu depuis Git, base de données managée
Hébergement front-end	Vercel	CDN mondial, HTTPS Let's Encrypt automatique, déploiement continu

Trois décisions techniques structurantes méritent d'être argumentées :

- L'ORM Django plutôt que du SQL brut, qui rend le code intrinsèquement résistant aux injections SQL tout en accélérant le développement grâce au système de migrations ;
- Les clés primaires UUID plutôt que des identifiants entiers auto-incrémentés, qui empêchent l'énumération de ressources par un attaquant (attaque IDOR : /api/audits/1, /api/audits/2…) ;
- Deux réponses distinctes selon la nature de l'atteinte à la sécurité sur les routes protégées : un code HTTP 404 lorsqu'un client authentifié tente d'accéder à une ressource d'un autre client (afin de ne pas révéler l'existence même de la ressource, logique IDOR), et un code HTTP 403 lorsqu'un utilisateur authentifié et légitime sur la ressource tente une action réservée à un rôle supérieur (logique RBAC, où l'existence de la ressource est déjà connue et seule l'action est interdite). Cette distinction, vérifiée dans le jeu d'essai (section 9) et les tests de sécurité (section 10.3), évite de traiter les deux mécanismes de contrôle d'accès de façon identique.

6.3 Flux de données : exemple de la soumission d'une demande d'audit

- Marie renseigne le formulaire dans l'application React (SPA).
- Le front-end envoie une requête POST /api/audits/ au format JSON, avec le jeton JWT dans l'en-tête Authorization: Bearer.
- Django valide la requête via un sérialiseur DRF puis la persiste en base via l'ORM.
- Django place dans la file Redis une tâche Celery asynchrone chargée d'appeler l'API EmailJS pour l'envoi de l'accusé de réception au client.

- Django répond 201 Created avec le détail de la demande créée, incluant sa référence unique.
- Karim (administrateur) voit apparaître la nouvelle demande dans son tableau de bord.

6.4 Accessibilité (RGAA / WCAG 2.1 niveau AA)

Le persona principal, Marie, non technophile, impose une exigence forte d'accessibilité. La plateforme vise la conformité au niveau AA du référentiel WCAG 2.1, décliné en France par le Référentiel général d'amélioration de l'accessibilité (RGAA), sur ses quatre principes : perceptible, utilisable, compréhensible, robuste.

Principe RGAA/WCAG	Mise en œuvre
Contraste minimal	Ratio **4,5:1** sur le texte courant**, 3:1** sur le texte large ; vérifié avec axe DevTools
Navigation au clavier	Tous les éléments interactifs accessibles à la touche Tab, à l'ordre de focus logique, à l'anneau de focus visible
Étiquettes ARIA	aria-label sur les badges de statut, role="meter" sur la jauge de score de sécurité
Formulaires	Chaque champ associé à une balise <label>, erreurs liées via aria-describedby
Structure sémantique	Balises header/nav/main/footer, hiérarchie de titres sans saut de niveau
Images et icônes	Attribut alt systématique, descriptif ou vide selon le caractère décoratif

L'accessibilité est contrôlée de façon automatisée (axe-core intégré aux tests Vitest, score Lighthouse Accessibilité supérieur ou égal à 90 exigé en intégration continue) et manuelle (navigation clavier complète testée sur les trois parcours critiques).

6.5 Budget de performance (Web Vitals) et éco-conception

L'objectif d'un parcours de moins de trois minutes pour un utilisateur non technicien impose une exigence de performance non négociable. La plateforme se fixe un budget de performance aligné sur les indicateurs Core Web Vitals de Google.

Indicateur	Cible	Signification
LCP (Largest Contentful Paint)	< 2,5 s	Temps d'affichage du plus grand élément visible
INP (Interaction to Next Paint)	< 200 ms	Latence de réponse aux interactions utilisateur
CLS (Cumulative Layout Shift)	< 0,1	Stabilité visuelle, absence de sauts de mise en page
TTFB (Time To First Byte)	< 600 ms	Délai serveur, couvre Django et la base de données
Poids JavaScript	< 200 ko compressé	Poids initial du bundle JavaScript
Score Lighthouse Performance	≥ 85/100	Score agrégé, calculé en intégration continue sur chaque pull request

Les stratégies d'optimisation retenues, qui relèvent également d'une démarche d'éco-conception des services numériques, incluent le découpage du code par route (*React.lazy* et Suspense) afin de ne transférer que le strict nécessaire, le *tree-shaking* automatique via Vite, le chargement différé (*lazy loading*) des images, la mise en cache HTTP des ressources statiques sur le CDN Vercel qui réduit les requêtes réseau redondantes, ainsi que la pagination

systématique des listes côté API (vingt éléments par défaut) et l'usage de *select_related* et *prefetch_related* côté Django pour éviter le problème des requêtes N+1 et limiter la charge serveur. Ces choix réduisent à la fois le temps de chargement perçu par un utilisateur non technophile et l'empreinte réseau et serveur de l'application.

7. Réalisations côté front-end

Cette section détaille les réalisations personnelles portant sur l'activité-type « Développer la partie front-end d'une application web ou web mobile sécurisée », en argumentant systématiquement les choix de sécurité retenus.

7.1 Installer et configurer son environnement de travail

L'environnement de développement front-end a été mis en place dès la première semaine, en tenant compte de l'environnement technique défini dans les spécifications (section 6). L'installation comprend Node.js 20, l'outil de compilation Vite, le gestionnaire de paquets npm, l'éditeur Visual Studio Code avec les extensions ESLint pour le contrôle de qualité de code, ainsi que le client Git pour le versionnement et le travail collaboratif avec James ROUSSEL.

Configuration du projet React/Vite (extrait de package.json) :

cyberaudit > frontend > package.json > ...

Tommy JOUHANS, il y a 2 mois | 2 authors (Tommy JOUHANS and one other)

```
{       Tommy JOUHANS, il y a 4 mois • Config environment setup …
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "description": "CyberAudit & Solutions — React frontend (Vite + Tailwind + EmailJS)",
  "main": "src/main.jsx",
  "author": "Tommy JOUHANS & James ROUSSEL",
  "license": "UNLICENSED",
  "keywords": [
    "cybersecurity",
    "audit",
    "react",
    "vite",
    "holberton"
  ],
  ▷Déboguer
  "scripts": {
    "dev": "vite",
    "lint": "eslint .",
    "fix": "eslint . --fix",
    "build": "vite build",
    "preview": "vite preview",
    "test": "vitest",
    "test:run": "vitest run",
    "test:coverage": "vitest run --coverage"
  },
```



```
"dependencies": {
  "@emailjs/browser": "^4.4.1",
  "@lhci/cli": "^0.15.1",
  "@tailwindcss/vite": "^4.3.0",
  "axios": "^1.9.0",
  "dompurify": "^3.4.8",
  "lucide-react": "^1.16.0",
  "react": "^19.2.6",
  "react-dom": "^19.2.6",
  "react-router-dom": "^7.1.1",
  "tailwindcss": "^4.3.0"
},
"devDependencies": {
  "@eslint/js": "^10.0.1",
  "@testing-library/jest-dom": "^6.6.3",
  "@testing-library/react": "^16.3.0",
  "@testing-library/user-event": "^14.6.1",
  "@types/react": "^19.2.14",
  "@types/react-dom": "^19.2.3",
  "@vitejs/plugin-react": "^6.0.1",
  "@vitest/coverage-v8": "^3.2.3",
  "daisyui": "^5.5.20",
  "eslint": "^10.3.0",
  "eslint-plugin-react-hooks": "^7.1.1",
  "eslint-plugin-react-refresh": "^0.5.2",
  "globals": "^17.6.0",
  "jsdom": "^26.1.0",
  "vite": "^8.0.12",
  "vitest": "^3.2.3"
}
}
```

Le contrôle de qualité de code repose sur ESLint (règles React et react-hooks) associé à Prettier pour le formatage automatique, exécutés en pré-commit et vérifiés en intégration continue. Le versionnement suit une convention de commits structurée (Conventional Commits : feat, fix, docs, test, refactor, chore), et chaque fonctionnalité est développée sur une branche dédiée (*feature/nom-de-la-fonctionnalité)* avant fusion par pull request revue par James ROUSSEL.

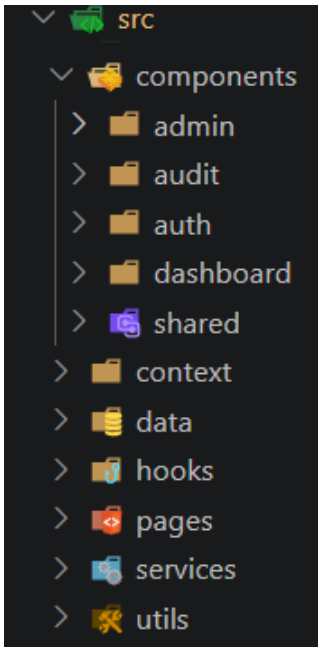
7.2 Maquetter les interfaces utilisateur

Le maquettage a été réalisé sur Figma à partir du recueil des besoins utilisateurs (section 3.2) et des user stories priorisées (section 5.2). Les onze écrans présentés en section 5.3 respectent une charte graphique cohérente (palette violette #5B21B6, typographie lisible, composants réutilisables), tiennent compte de l'expérience utilisateur du persona Marie et intègrent, dès la conception, les exigences de sécurisation : aucun mot de passe n'est jamais affiché en clair dans les maquettes, et les formulaires prévoient systématiquement un espace dédié à l'affichage des messages d'erreur de validation.

Le schéma d'enchaînement des maquettes, présenté en section 5.3, modélise la navigation entre les onze écrans et a été validé avant le début du développement des interfaces statiques, conformément à la démarche attendue par le référentiel.

7.3 Arborescence des composants React

L'arborescence ci-dessous illustre la décomposition de l'application en composants réutilisables. Le composant *StatusBadge*, par exemple, est partagé entre le tableau de bord client, le tableau de bord administrateur et la vue de détail d'une demande, ce qui garantit une cohérence visuelle et limite la duplication de code.

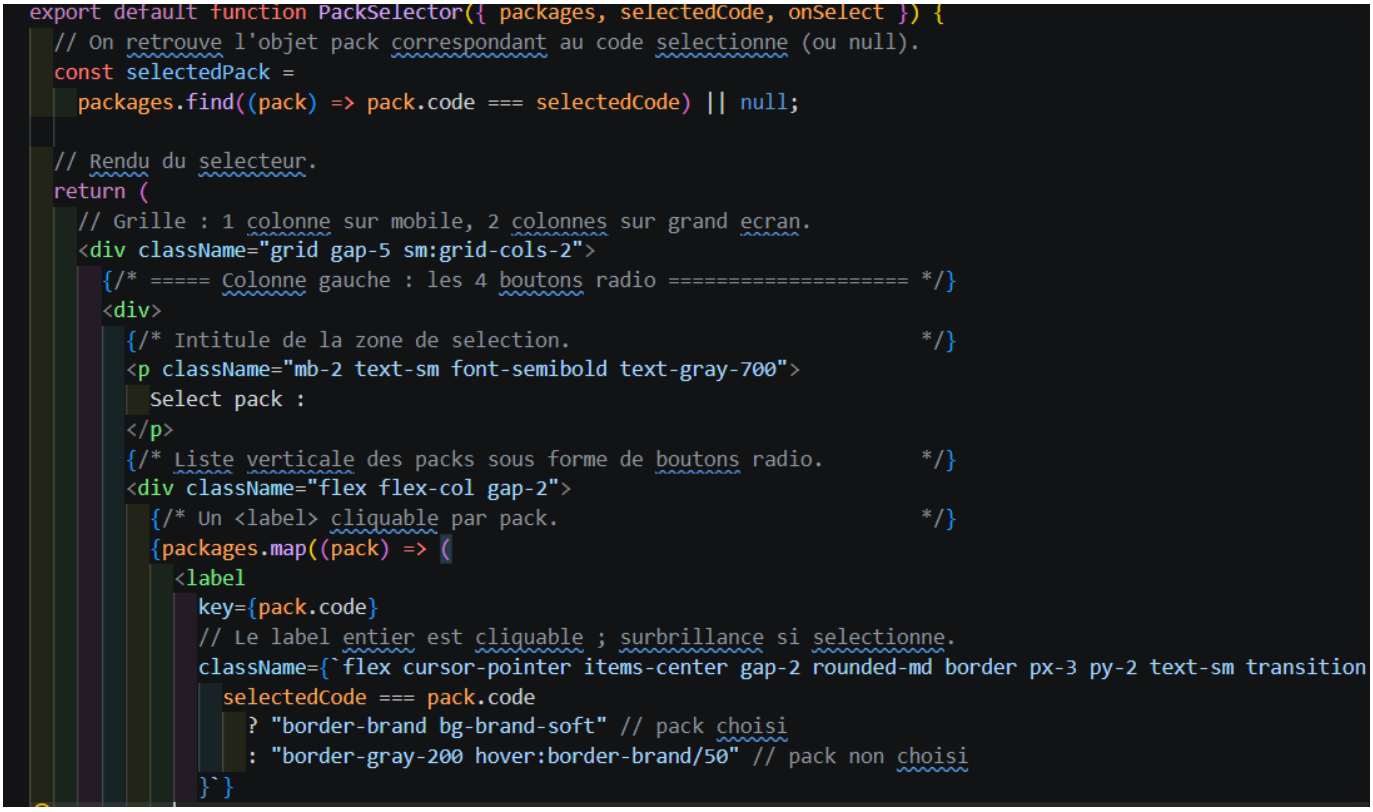



```
AdminDashboard.jsx
AuditRequestForm.jsx, PackSelector.jsx
LoginForm.jsx, ProtectedRoute.jsx, RegisterForm.jsx
ClientDashboard.jsx, StatCard.jsx, StatusBadge.jsx
Header.jsx, Sidebar.jsx, Footer.jsx, ErrorBoundary.jsx, Logo.jsx, …
AuthContext.jsx
notifications.json, packages.json, reports.json, requests.json, …
useAuth.js
HomePage.jsx, LoginPage.jsx, RegisterPage.jsx, DashboardPage.jsx, …
api.js, authService.js, dataService.js, emailService.js
sanitize.js, validators.js
```

7.4 Réaliser les interfaces utilisateur statiques

Les interfaces statiques ont été développées en HTML sémantique généré par React (JSX) et stylées avec Tailwind CSS 4, en respectant la charte graphique définie sur les maquettes. L'adaptabilité aux différents types de supports (ordinateur, tablette, mobile) a été traitée dès cette étape, à l'aide des classes utilitaires responsives de Tailwind, selon une approche « mobile first ».

Extrait de composant statique : carte de formule d'audit (*PackSelector.jsx*) :



```jsx
export default function PackSelector({ packages, selectedCode, onSelect }) {
  // On retrouve l'objet pack correspondant au code selectionne (ou null).
  const selectedPack =
    packages.find((pack) => pack.code === selectedCode) || null;

  // Rendu du selecteur.
  return (
    // Grille : 1 colonne sur mobile, 2 colonnes sur grand ecran.
    <div className="grid gap-5 sm:grid-cols-2">
      {/* ===== Colonne gauche : les 4 boutons radio ==================== */}
      <div>
        {/* Intitule de la zone de selection.                          */}
        <p className="mb-2 text-sm font-semibold text-gray-700">
          Select pack :
        </p>
        {/* Liste verticale des packs sous forme de boutons radio.     */}
        <div className="flex flex-col gap-2">
          {/* Un <label> cliquable par pack.                           */}
          {packages.map((pack) => (
            <label
              key={pack.code}
              // Le label entier est cliquable ; surbrillance si selectionne.
              className={`flex cursor-pointer items-center gap-2 rounded-md border px-3 py-2 text-sm transition
                selectedCode === pack.code
                  ? "border-brand bg-brand-soft" // pack choisi
                  : "border-gray-200 hover:border-brand/50" // pack non choisi
              }`}
```

Les règles de validation des formulaires statiques ont été précisées dès cette étape (voir section 5.3, écran 2). La documentation du code a été rédigée en français et en anglais selon le contexte, conformément à l'exigence de niveau B1 du CECRL pour la compréhension écrite de l'anglais professionnel. Le référencement des pages publiques a été optimisé via des balises meta et un fichier robots.txt.

7.5 Développer la partie dynamique des interfaces utilisateur

La partie dynamique repose sur les hooks React et un contexte d'authentification global (*AuthContext*), qui gère l'état de connexion, le stockage des jetons JWT et la vérification des rôles. Le composant *ProtectedRoute* encapsule cette logique pour protéger l'accès aux pages selon le rôle de l'utilisateur connecté.

```jsx
// ProtectedRoute recoit la prop "allowedRoles" : la liste des roles autorises.
// Valeur par defaut [] = aucun role specifique exige (juste etre connecte).
export default function ProtectedRoute({ allowedRoles = [] }) {
  // On lit l'etat d'authentification : connecte ? pret ? quel utilisateur ?
  const { isAuthenticated, ready, user } = useAuth();
  // On recupere l'URL demandee, pour pouvoir y revenir apres connexion.
  const location = useLocation();

  // Tant que la session initiale n'est pas verifiee, on affiche un message
  // d'attente : cela evite une redirection erronee au premier rendu.
  if (!ready) {
    return (
      <div className="flex min-h-[50vh] items-center justify-center text-brand">
        Loading...
      </div>
    );
  }

  // Cas 1 : utilisateur NON connecte -> redirection vers la page de connexion.
  // "state" memorise la page demandee ; "replace" evite d'empiler l'historique.
  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  // Cas 2 : utilisateur connecte mais dont le role n'est pas autorise ici.
  // (allowedRoles non vide ET ne contient pas le role de l'utilisateur).
  if (allowedRoles.length > 0 && !allowedRoles.includes(user.role)) {
    // Acces interdit pour ce role -> on renvoie vers la page d'accueil.
    return <Navigate to="/" replace />;
  }

  // Cas 3 : utilisateur connecte ET autorise -> on affiche la route enfant.
  return <Outlet />;
}
```

L'appel à l'API REST est centralisé dans une instance Axios unique (api.js), qui injecte automatiquement le jeton JWT dans l'en-tête Authorization de chaque requête authentifiée, et intercepte les réponses 401 pour tenter un rafraîchissement automatique du jeton avant de rediriger vers la page de connexion en cas d'échec.

Le tableau de bord client (*ClientDashboard*) illustre la partie dynamique la plus représentative de l'application : au chargement, un *hook* personnalisé useAudits interroge l'API paginée, affiche un indicateur de chargement, puis restitue la liste des demandes avec un composant *StatusBadge* dont la couleur varie selon le statut. Un clic sur une demande déclenche la navigation vers sa vue de détail sans rechargement de page, grâce à React Router.

7.6 Sécurité de la partie front-end

La sécurité côté client a été traitée comme une exigence transversale et non comme une étape a posteriori. Quatre mesures principales ont été mises en œuvre et argumentées :

Protection contre les attaques XSS

Toute donnée saisie par un utilisateur et réaffichée dans l'interface (notes de périmètre d'une demande d'audit, contenu Markdown des modules de formation) est systématiquement assainie par la bibliothèque DOMPurify avant insertion dans le DOM.

En-têtes de sécurité HTTP

Le déploiement Vercel applique des en-têtes de sécurité sur chaque réponse : X-Content-Type-Options: nosniff, X-Frame-Options: DENY (anti-clickjacking), une politique de sécurité de contenu (CSP) restrictive, et Strict-Transport-Security (HSTS) qui impose l'usage exclusif du protocole HTTPS.

Gestion sécurisée des jetons d'authentification

Les jetons d'accès et de rafraîchissement JWT sont stockés en *sessionStorage* (isolé par onglet) plutôt qu'en cookie non sécurisé, et jamais journalisés côté client. Le jeton d'accès a une durée de vie courte de soixante minutes, tandis que le jeton de rafraîchissement, valable sept jours, fait l'objet d'une rotation à chaque utilisation côté serveur (section 8.4).

Validation des entrées côté client

Bien que la validation faisant foi soit systématiquement réalisée côté serveur (section 8.4), une validation côté client est appliquée en amont pour améliorer l'expérience utilisateur : le module *validators.js* centralise les règles de validation et est couvert par des tests unitaires (section 10.2).

8. Réalisations côté back-end

Bien que le développement back-end ait été principalement conduit par James ROUSSEL, cette section documente, conformément aux exigences du dossier de projet, l'activité-type « Développer la partie back-end d'une application web ou web mobile sécurisée » telle que conçue et mise en œuvre collectivement pour le projet. Les choix de modélisation ont été définis en binôme lors des sprints de conception (S1-S2), ma contribution personnelle ayant porté sur la définition des contrats d'interface consommés par le front-end, la co-définition des exigences de sécurité de l'API (section 6.2), et la relecture de sécurité croisée imposée par nos règles de pull request (voir aussi le dossier professionnel joint, activité-type 2).

8.1 Mettre en place une base de données relationnelle

Le schéma conceptuel de données a été élaboré à partir de l'expression des besoins (section 3.2), en identifiant sept entités principales : l'utilisateur (client ou administrateur), la formule d'audit, la demande d'audit, le rapport d'audit, le module de formation, la progression de formation et la notification. Les relations entre ces entités reflètent le cycle de vie métier de l'application : un utilisateur soumet plusieurs demandes, chaque demande référence une formule et donne lieu à au plus un rapport, et chaque notification est rattachée à un utilisateur et le cas échéant à une demande.

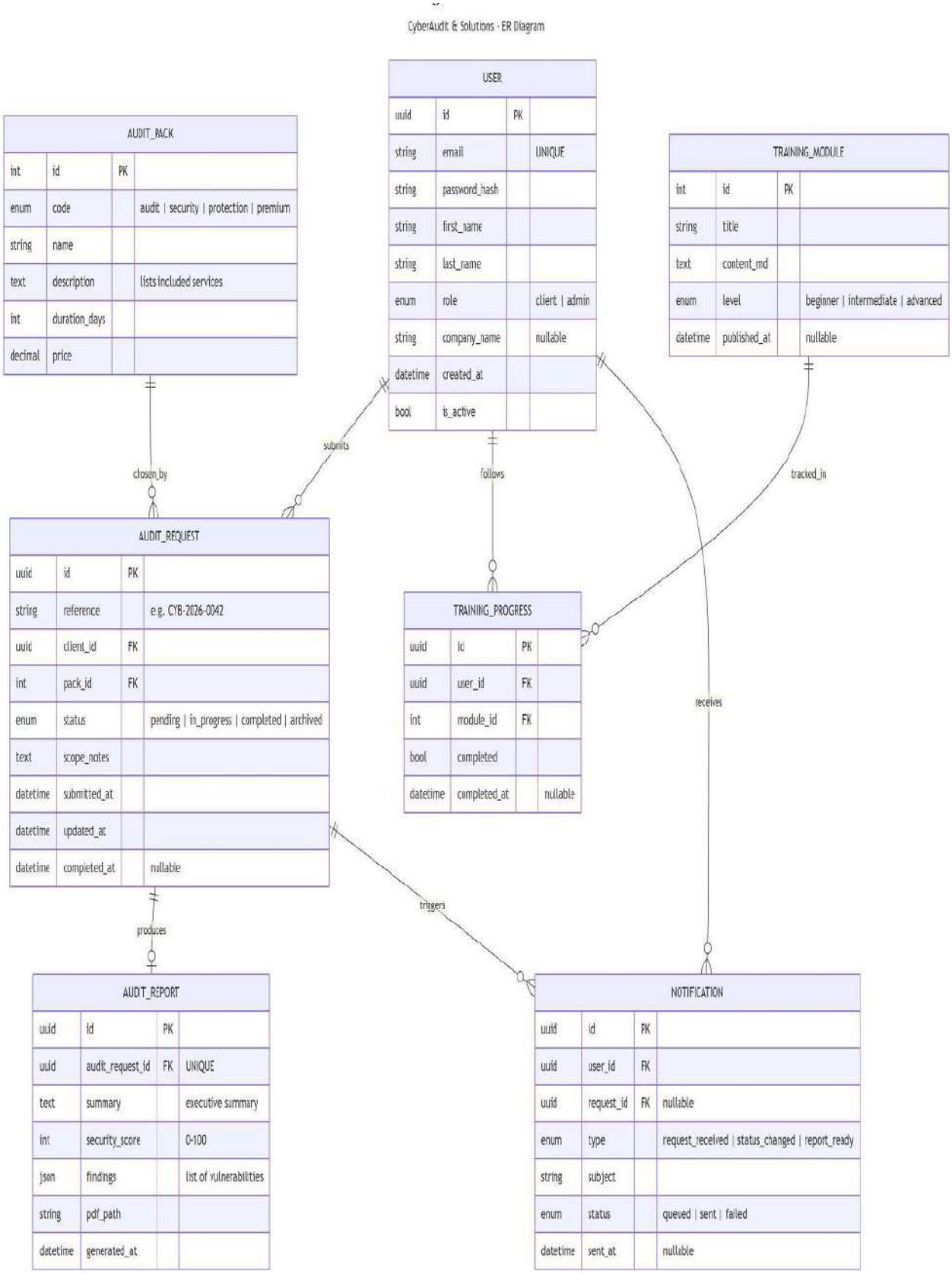


Schéma physique de la base de données relationnelle PostgreSQL composées de sept tables, clés primaires UUID, clés étrangères et index de performance.

Entité	Champs clés	Relations
USER	id (UUID), email (unique), password_hash, role, company_name	1..N vers AUDIT_REQUEST, NOTIFICATION
AUDIT_PACK	id, code, name, duration_days, price	1..N vers AUDIT_REQUEST
AUDIT_REQUEST	id (UUID), reference (unique), status, submitted_at	N..1 vers USER et AUDIT_PACK ; 1..1 vers AUDIT_REPORT
AUDIT_REPORT	id (UUID), audit_request_id (unique), grade, security_score, findings (JSON)	1..1 vers AUDIT_REQUEST
TRAINING_MODULE	id, title, content_md, level	1..N vers TRAINING_PROGRESS
TRAINING_PROGRESS	id, user_id, module_id, completed	N..1 vers USER et TRAINING_MODULE
NOTIFICATION	id (UUID), user_id, type, status, sent_at	N..1 vers USER et AUDIT_REQUEST

Quatre index ont été ajoutés pour garantir la performance des requêtes les plus fréquentes : un index unique sur l'adresse électronique de l'utilisateur, un index sur le statut d'une demande (filtrage administrateur fréquent), un index composite sur le client et la date de soumission (tableau de bord client), et un index unique sur la relation demande-rapport, garantissant l'unicité d'un rapport par demande.

8.2 Développer des composants d'accès aux données SQL et NoSQL

L'accès aux données relationnelles est intégralement réalisé via l'ORM de Django, qui génère des requêtes SQL paramétrées et protège nativement l'application contre les injections SQL, aucune requête brute n'étant utilisée dans le projet. Au-delà du modèle relationnel, le projet exploite une base de données NoSQL de type clé-valeur : Redis, utilisée comme courtier de messages (broker) pour la file de tâches asynchrones Celery. Chaque tâche différée (génération de rapport PDF, envoi de courriel via EmailJS) est sérialisée et placée dans une file Redis, consommée par un worker Celery indépendant du processus web principal.

8.3 Développer des composants métier côté serveur

La logique métier est séparé des modèles de données et regroupée dans des modules de services dédiés, selon le principe de responsabilité unique. Le modèle *AuditRequest* expose par exemple une méthode métier de génération de référence unique, garantissant l'absence de collision même en cas de créations concurrentes grâce à un verrou de ligne (*select_for_update*).

```python
@classmethod
def _generate_reference(cls):
    """Génère DOSSIER-YYYY-NNNN de manière atomique (anti-collision)."""
    year = datetime.now().year
    prefix = f"DOSSIER-{year}-"
    with transaction.atomic():
        last = (
            cls.objects.select_for_update()
            .filter(reference__startswith=prefix)
            .order_by("-reference")
            .first()
        )
        next_num = (int(last.reference.split("-")[-1]) + 1) if last else 1
    return f"{prefix}{next_num:04d}"
```

Le composant métier de génération du rapport PDF illustre le traitement asynchrone côté serveur : à la demande de l'administrateur, une tâche Celery restitue le gabarit HTML du rapport avec les données de la demande, le convertit en PDF via WeasyPrint, l'enregistre, puis déclenche l'envoi d'une notification par courriel au client via EmailJS. Une politique de nouvelle tentative automatique (jusqu'à trois essais, délai croissant de dix, trente puis quatre-vingt-dix secondes) sécurise ce traitement contre les échecs transitoires.

8.4 Sécurité de la partie back-end

Authentification et gestion des sessions

L'authentification repose sur SimpleJWT : un jeton d'accès de courte durée (soixante minutes) et un jeton de rafraîchissement de sept jours, avec rotation systématique du jeton de rafraîchissement à chaque utilisation et mise en liste noire (blacklist) du jeton précédent. Les mots de passe sont hachés avec l'algorithme PBKDF2 (paramétrage par défaut de Django), jamais stockés en clair.

Contrôle d'accès par rôle (RBAC)

Deux niveaux de défense protègent les routes sensibles : côté front-end, le composant *ProtectedRoute* (section 7.5) empêche l'affichage d'une page non autorisée ; côté back-end, des classes de permission DRF personnalisées (*IsAdmin, IsAdminOrOwner*) sont vérifiées sur chaque vue avant tout traitement. Conformément à la distinction posée en section 6.2, un client tentant d'accéder à la demande d'un autre client reçoit un 404 (dissimulation de l'existence de la ressource), tandis qu'un client tentant une action réservée à l'administrateur sur une ressource dont il connaît déjà l'existence reçoit un 403 (action interdite par le rôle).

Limitation du débit (rate limiting)

Les points d'entrée sensibles aux attaques par force brute (connexion, réinitialisation de mot de passe) sont limités à cinq requêtes par minute et par adresse IP, à l'aide du décorateur django-ratelimit.

Protection contre les injections et validation des entrées

L'usage exclusif de l'ORM Django élimine par construction le risque d'injection SQL. Chaque requête entrante est validée par un sérialiseur DRF avant tout traitement métier. Les champs de texte libre ne sont jamais interprétés comme du HTML côté serveur, ce qui élimine le risque d'injection de code exécutable au niveau du gabarit du rapport PDF.

En-têtes de sécurité et transport

L'ensemble des échanges transite exclusivement en HTTPS, avec certificats renouvelés automatiquement (Let's Encrypt). Un point de contrôle applicatif GET /api/health/ vérifie en continu la disponibilité de la base de données, du courtier Redis et du service EmailJS.

8.5 Documenter le déploiement d'une application dynamique web ou web mobile

Le déploiement de l'application suit une démarche d'intégration et de livraison continues (CI/CD), automatisée par GitHub Actions à chaque poussée de code sur les branches main et develop. Le pipeline exécute successivement l'analyse statique du code (ruff, black côté back-end ; ESLint côté front-end), la suite de tests avec couverture minimale exigée, puis la construction et le déploiement automatique en cas de succès.

```yaml
jobs:
  test:
    defaults:
      run:
        working-directory: cyberaudit/backend
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: System deps (WeasyPrint)
        run: |
          sudo apt-get update
          sudo apt-get install -y \
            libcairo2 libpango-1.0-0 libpangoft2-1.0-0 \
            libharfbuzz0b libfontconfig1 fonts-liberation

      - name: Install Python deps
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Lint (ruff)
        run: ruff check apps/ config/ --ignore E501

      - name: Format check (black)
        run: black --check apps/ config/

      - name: Tests with coverage
        run: pytest --cov=apps --cov-report=term-missing --cov-fail-under=40
```

Le seuil de couverture minimal exigé par le pipeline (40 %) est un seuil plancher de non-régression ; la couverture effectivement atteinte en fin de projet est de 91 % côté back-end (section 10.1).

Le déploiement en production du back-end est orchestré par Railway à partir d'un conteneur Docker et d'un fichier Procfile qui définit deux processus : un processus web appliquant automatiquement les migrations de base de données puis démarrant le serveur applicatif Gunicorn, et un processus worker exécutant le worker Celery.

```
cyberaudit > backend > Procfile
       James0812, il y a 3 mois | 2 authors (You and one other)
1      web: python manage.py migrate --noinput && python manage.py setup_admin \
2       && gunicorn config.wsgi --bind 0.0.0.0:$PORT --log-file - --access-logfile - --workers 2
3      worker: celery -A config worker --loglevel=info --concurrency=2
```

Le front-end est déployé sur Vercel, qui reconstruit automatiquement l'application à chaque poussée sur la branche principale et applique les en-têtes de sécurité HTTP définis dans vercel.json (section 7.6). Les variables sensibles (clés secrètes, identifiants de base de données, clés d'API) sont exclusivement gérées via les variables d'environnement des deux plateformes et ne figurent jamais dans le dépôt de code source, protégées par un fichier « *.gitignore* » et fournies en exemple via un fichier « *.env.example* » non renseigné.

Une sauvegarde manuelle de la base de données de production est déclenchée avant chaque mise en production majeure (railway db snapshot create), avec une rétention de quatorze jours, permettant un retour arrière en moins de cinq minutes en cas d'anomalie détectée après déploiement.

Les diagrammes de conception complémentaires (diagrammes de classes back-end et front-end, diagrammes de séquence) sont présentés en Annexe 5, afin de ne pas alourdir le corps du dossier.

9. Jeu d'essai

Conformément à la méthodologie retenue, le jeu d'essai a été élaboré sur la fonctionnalité la plus représentative de l'application : la soumission d'une demande d'audit par un client PME, qui mobilise l'ensemble de la chaîne technique (front-end, API sécurisée, base de données, tâche asynchrone). Il couvre les données en entrée, les données attendues, les données réellement obtenues, ainsi que l'analyse des écarts constatés.

Scénario	Données en entrée	Résultat attendu	Résultat obtenu	Écart
Soumission valide	pack_id=2, scope_notes="Audit du serveur de fichiers et de 12 postes Windows.", JWT valide	201 Created, statut=pending, référence DOSSIER-2026-NNNN générée	201 Created, référence DOSSIER-2026-0001 générée, accusé de réception envoyé via EmailJS	Aucun
Champ obligatoire manquant	pack_id absent	400 Bad Request avec message d'erreur explicite	400 Bad Request, {"pack_id": ["Ce champ est obligatoire."]}	Aucun
Jeton JWT absent	Aucun en-tête Authorization	401 Unauthorized	401 Unauthorized	Aucun
Jeton JWT expiré	JWT expiré depuis 2 heures	401 puis rafraîchissement côté client	401 Unauthorized, rafraîchissement automatique puis rejeu de la requête réussi	Aucun
Tentative d'injection XSS	scope_notes="<script>alert(1)</script>"	Contenu stocké tel quel côté serveur, neutralisé à l'affichage côté client	Contenu stocké en base, assaini par DOMPurify à l'affichage, script non exécuté	Aucun
Accès à la demande d'un autre client (IDOR)	GET /api/audits/{id-B}/ avec le jeton du client A	404 Not Found (dissimulation de l'existence de la ressource)	404 Not Found	Aucun
Élévation de privilège (RBAC)	PATCH /api/audits/{id}/ avec le jeton d'un client sur sa propre ressource	403 Forbidden (action réservée à l'administrateur)	403 Forbidden	Aucun
Formule inexistante	pack_id=999	400 Bad Request	400 Bad Request, {"pack_id": ["Objet invalide."]}	Aucun
Requêtes répétées (force brute)	6 tentatives de connexion en moins d'une minute	Blocage à la 6e tentative (429 Too Many Requests)	429 Too Many Requests déclenché à la 6e tentative	Aucun

Aucun écart n'a été constaté entre le résultat attendu et le résultat obtenu sur l'ensemble des neuf scénarios du jeu d'essai, qui couvre à la fois le chemin nominal, les cas d'erreur de validation, et les cas d'attaque. Les scénarios d'accès non autorisé (IDOR) et d'élévation de privilège (RBAC) illustrent volontairement les deux réponses HTTP distinctes justifiées en section 6.2. Ce jeu d'essai a été rejoué manuellement via la collection Postman du projet, puis automatisé sous forme de tests d'intégration pérennes (section 10.2).

10. Stratégie de tests et qualité

La stratégie de tests suit une pyramide classique : une large base de tests unitaires rapides, une couche intermédiaire de tests d'intégration API et base de données, et un sommet réduit de tests de bout en bout sur les parcours critiques.

10.1 Tests back-end (pytest)

Application	Scénarios couverts
accounts	Modèle utilisateur, inscription, connexion, profil, déconnexion, changement de mot de passe
audits	Génération automatique de référence, unicité, CRUD avec RBAC client/admin, archivage
reports	Calcul de la note (A/B+/B/C/F), accès au rapport (propriétaire/autre client/admin)
training	Modules publiés/non publiés, démarrage et achèvement idempotents
notifications	Envoi et journalisation des e-mails transactionnels (EmailJS)

L'exécution de la suite complète (commande locale : pytest --cov=apps --cov-report=term-missing -v) donne le résultat suivant, confirmé de façon reproductible sur plusieurs exécutions consécutives en local et en intégration continue GitHub Actions : 74 tests, tous passants (« 74 passed »), pour une couverture de code de 91 % (1 242 instructions dont 114 non couvertes, rapport détaillé exporté en HTML via --cov-report=html). Ce résultat est supérieur au seuil plancher de non-régression imposé par le pipeline (40 %, à voir section 8.5). Les instructions non couvertes se concentrent principalement sur des branches d'erreur secondaires et sur l'application notifications, identifiées comme axe d'amélioration en section 12.1.

10.2 Tests front-end (Vitest et React Testing Library)

Module testé	Fichier de test	Tests
validators.js, sanitize.js	validators.test.js, sanitize.test.js	27
StatCard, StatusBadge	StatCard.test.jsx, StatusBadge.test.jsx	11
useAuth, ProtectedRoute	useAuth.test.jsx, ProtectedRoute.test.jsx	11
dataService (couche API)	dataService.test.js	7
DashboardPage	DashboardPage.test.jsx	2
TrainingPage	TrainingPage.test.jsx	4

Soit un total de 9 fichiers de test et 62 tests front-end, exécutés via npm test run:coverage et intégrés au pipeline d'intégration continue front-end. La couverture front-end reste volontairement inégale à l'issue du projet : elle est proche de 100 % sur les modules jugés les plus critiques (fonctions utilitaires de validation et d'assainissement anti-XSS, hooks d'authentification, garde d'accès par rôle), mais les pages métier les plus complexes (formulaire de demande d'audit, tableaux de bord administrateur, pages d'authentification) et les composants de mise en page ne disposent pas encore de tests automatisés dédiés, la priorité ayant été donnée en fin de projet à la couverture back-end et aux tests de sécurité manuels (section 10.3). Ce point est repris honnêtement comme axe d'amélioration en section 12.1.

10.3 Tests de sécurité manuels

Test	Méthode	Résultat attendu
Injection XSS persistante	<script>alert(1)</script> dans scope_notes	Contenu assaini, script non exécuté
Injection SQL	' OR 1=1 -- dans le champ e-mail de connexion	Aucune fuite, requête paramétrée par l'ORM
Force brute sur la connexion	Automatisation via Postman, 10 tentatives rapides	Blocage à la 6e tentative (429)
Falsification de jeton JWT	Modification manuelle du rôle dans le payload décodé	Signature invalide, 401 Unauthorized
Élévation de privilège (RBAC)	Client tentant PATCH /api/audits/{id}/	403 Forbidden
Énumération de ressources (IDOR)	GET /api/audits/{id-B}/ avec le jeton du client A	404 Not Found

L'audit manuel de sécurité mené en semaines 7-8 n'a révélé aucune faille exploitable sur les six scénarios testés, confirmant l'efficacité des mesures de sécurisation mises en œuvre dès la conception. Les résultats confirment notamment la distinction attendue entre 403 (RBAC) et 404 (IDOR) posée en section 6.2.

10.4 Tests d'accessibilité et de performance

L'accessibilité est vérifiée automatiquement par axe-core sur les composants critiques (LoginForm, AuditRequestForm, RequestTable) intégré aux tests Vitest, et par un score Lighthouse Accessibilité supérieur ou égal à 90 exigé en intégration continue sur les quatre pages principales. La performance est mesurée par Lighthouse CI sur chaque pull request, avec un score Performance cible supérieur ou égal à 85/100 conformément au budget défini en section 6.5.

11. Veille technologique et sécuritaire

Une veille continue a été assurée tout au long des douze semaines du projet, structurée autour de trois axes : les vulnérabilités de sécurité applicative, les évolutions techniques de l'écosystème utilisé, et les bonnes pratiques d'accessibilité et de performance.

11.1 Sources suivies

- Le guide OWASP Top 10 en Annexe 7, référence des dix familles de vulnérabilités web les plus critiques, utilisé comme grille de contrôle avant chaque mise en production.
- Les recommandations et guides de l'Agence nationale de la sécurité des systèmes d'information (ANSSI), notamment le guide d'hygiène informatique et les recommandations de sécurisation d'un site web.
- Les avis de sécurité CERT-FR (cert.ssi.gouv.fr) pour le suivi des vulnérabilités publiées (CVE) touchant les dépendances du projet (Django, React, bibliothèques npm et PyPI).
- Les alertes automatiques GitHub Dependabot, qui signalent les dépendances vulnérables directement dans le dépôt du projet.
- La documentation officielle des frameworks utilisés (Django, Django REST Framework, React).

11.2 Méthode et traçabilité

La veille a été menée de façon hebdomadaire, en parallèle des sprints de développement, avec un temps dédié le vendredi lors du point de synthèse. Chaque élément pertinent identifié était consigné dans l'espace Notion du projet, puis discuté en binôme pour évaluer sa pertinence et, le cas échéant, planifier une action corrective.

11.3 Exemple concret de vulnérabilité suivie et traitée

Lors du sprint 5, la veille CERT-FR a signalé un avis de sécurité concernant une version antérieure de Django REST Framework, liée à une désérialisation incorrecte pouvant exposer certains champs sensibles dans les réponses API. Le projet utilisant déjà une version corrigée du framework (DRF 3.17), aucune action immédiate n'a été nécessaire, mais la version a été explicitement figée dans requirements.txt et une alerte Dependabot activée pour prévenir toute régression lors d'une mise à jour future. Cet épisode illustre la démarche structurée de veille attendue par le référentiel : identification de la source, analyse de l'exposition réelle du projet, décision argumentée, et traçabilité de la décision.

12. Conclusion

Le projet CyberAudit & Solutions a permis de concevoir et de développer, en douze semaines et en binôme, une application web complète répondant à un besoin réel et documenté : l'accès des petites et moyennes entreprises françaises à des services de cybersécurité simples et abordables. Sur le plan personnel, ce projet a été l'occasion de mettre en œuvre l'intégralité des compétences de l'activité-type front-end du titre professionnel Développeur Web et Web Mobile, du maquettage des interfaces jusqu'à leur développement dynamique sécurisé, en passant par la configuration d'un environnement de travail professionnel et l'intégration continue.

La contrainte de délai fixe, la nécessité de synchroniser en permanence les contrats d'interface avec un développeur back-end, et l'exigence transversale de sécurité à chaque étape ont constitué les principales difficultés du projet. Elles ont été surmontées par une démarche agile disciplinée, des revues de code croisées systématiques et une veille technologique continue, qui ont permis de livrer 95 % du périmètre initialement planifié, avec zéro bogue ouvert à l'issue de chaque sprint et une couverture de tests automatisés, soit 74 tests back-end (91 % de couverture) et 62 tests front-end, exécutés à chaque poussée de code via GitHub Actions.

Ce projet m'a permis de consolider une véritable culture de la sécurité applicative « by design », de gagner en autonomie sur l'écosystème React et son intégration à une API REST sécurisée, et de mesurer concrètement l'importance de l'accessibilité, de la performance et de l'éco-conception dans la conception d'une interface destinée à un public non technophile.

12.1 Perspectives d'évolution

- Mise en place d'un tableau de bord temps réel par WebSocket, en remplacement du polling actuel toutes les trente secondes.
- Calcul automatique d'un score de sécurité assisté par intelligence artificielle à partir des réponses au questionnaire d'audit.
- Déclinaison de l'application en application mobile native (iOS/Android).
- Gamification des modules de sensibilisation à la cybersécurité, et achèvement de leur développement applicatif complet (voir section 5.3, écran 7).
- Mise en place d'une authentification déléguée OAuth2 (Google, LinkedIn) pour simplifier l'inscription.
- Journal d'audit applicatif complet (table AUDIT_LOG) pour une traçabilité renforcée conforme RGPD des actions sensibles.

13. Remerciements

Je tiens à remercier l'ensemble des personnes qui ont contribué, de près ou de loin, à la réussite de ce projet portfolio réalisé dans le cadre de ma formation au Titre RNCP de Niveau 5 de Développeur Web et Web Mobile.

Mes remerciements vont en premier lieu à James ROUSSEL, mon binôme sur ce projet, pour la qualité de notre collaboration, la rigueur de nos échanges techniques et la complémentarité de nos compétences front-end et back-end tout au long des douze semaines de développement.

Je remercie également l'équipe pédagogique de Holberton School Dijon, notamment mon référent technique et mon relecteur de projet, pour leurs conseils, leur disponibilité et la qualité de leurs retours lors des revues de code et des points d'avancement hebdomadaires.

Enfin, je remercie mes proches pour leur soutien tout au long de cette formation intensive, ainsi que l'ensemble de la promotion pour l'entraide et l'émulation collective qui ont accompagné la préparation de ce dossier.

14. Annexes

Annexe 1 : Glossaire technique

Terme	Définition
API REST	Interface de programmation respectant les principes de l'architecture REST, utilisant le protocole HTTP
JWT (JSON Web Token)	Jeton d'authentification sans état, signé, transmis dans l'en-tête HTTP Authorization
RBAC	Role-Based Access Control, contrôle d'accès basé sur le rôle de l'utilisateur
ORM	Object-Relational Mapping, correspondance entre objets du code et tables d'une base relationnelle
CI/CD	Intégration continue et livraison/déploiement continus
XSS (Cross-Site Scripting)	Faille permettant l'injection de code exécutable dans une page consultée par d'autres utilisateurs
CSRF	Falsification de requête intersites, attaque exploitant la session active d'un utilisateur
RGAA	Référentiel général d'amélioration de l'accessibilité, déclinaison française du WCAG
UUID	Identifiant universel unique, non séquentiel, utilisé pour prévenir l'énumération de ressources
SPA (Single Page Application)	Application web fonctionnant sur une page unique, sans rechargement complet
IDOR	Insecure Direct Object Reference : accès direct non autorisé à une ressource via son identifiant

Annexe 2 : Correspondance compétences REAC / sections du dossier

Cette annexe reprend, à titre de synthèse rapide pour le jury, le tableau de correspondance entre les huit compétences professionnelles du référentiel DWWM et les sections du dossier où chacune est mise en œuvre et argumentée, déjà présenté et commenté en section 2.

Activité-type	Compétence professionnelle	Section
Front-end	Installer et configurer son environnement de travail	7.1
Front-end	Maquetter des interfaces utilisateur web ou web mobile	7.2
Front-end	Réaliser des interfaces utilisateur statiques web ou web mobile	7.4
Front-end	Développer la partie dynamique des interfaces utilisateur	7.5
Back-end	Mettre en place une base de données relationnelle	8.1
Back-end	Développer des composants d'accès aux données SQL et NoSQL	8.2
Back-end	Développer des composants métier côté serveur	8.3
Back-end	Documenter le déploiement d'une application dynamique	8.5

Annexe 3 : Matrice des risques complète

La matrice des risques du projet, présentée et détaillée en section 4.5, est reproduite ci-dessous à titre de référence rapide pour le jury.

Risque identifié	Probabilité	Impact	Mesure de mitigation
Retard sur l'intégration front/back par divergence des contrats d'API	Moyenne	Élevé	Définition des schémas JSON en amont (S1), localStorage côté front
Faille de sécurité non détectée avant la mise en production	Moyenne	Élevé	Tests de sécurité manuels dédiés (S7-S10), revue de code croisée, veille OWASP
Non-respect du délai de douze semaines	Faible	Élevé	Sprints avec priorisation MoSCoW, points hebdomadaires de replanification
Indisponibilité d'un des deux développeurs	Faible	Moyen	Documentation continue sur Notion, séparation claire des périmètres
Dépassement du budget de performance (Web Vitals)	Faible	Moyen	Lighthouse CI intégré au pipeline, budget de performance défini dès la conception

Annexe 4 : Bibliographie et sources

Source	Contenu	Référence
ANSSI	Panorama de la cybermenace 2023, 34 % des victimes de rançongiciels sont des TPE/PME/ETI	cyber.gouv.fr
CESIN	Baromètre annuel de la cybersécurité	cesin.fr
ENISA	Rapport sur le paysage des menaces 2023	enisa.europa.eu
RGPD	Texte officiel du Règlement général sur la protection des données	gdpr.eu
Directive NIS2	Obligations de cybersécurité au niveau de l'Union européenne	digital-strategy.ec.europa.eu
OWASP	OWASP Top 10 : référentiel des vulnérabilités web critiques	owasp.org
W3C	Web Content Accessibility Guidelines (WCAG) 2.1	w3.org/WAI/WCAG21
RGAA	Référentiel général d'amélioration de l'accessibilité	accessibilite.numerique.gouv.fr
Documentation Django	Documentation officielle du framework Django 5.1	docs.djangoproject.com
Documentation React	Documentation officielle de React 19	react.dev
Documentation DRF	Documentation officielle de Django REST Framework	django-rest-framework.org
CERT-FR	Avis et alertes de sécurité	cert.ssi.gouv.fr

Annexe 5 : Diagrammes de conception complémentaires

Les diagrammes ci-dessous complètent la documentation technique du projet, présentée en section 8 : le diagramme de classes back-end modélise les entités métier et leurs méthodes, le diagramme de classes front-end modélise l'organisation des composants React, et le diagramme de séquence illustre le scénario d'interaction le plus représentatif de l'application.

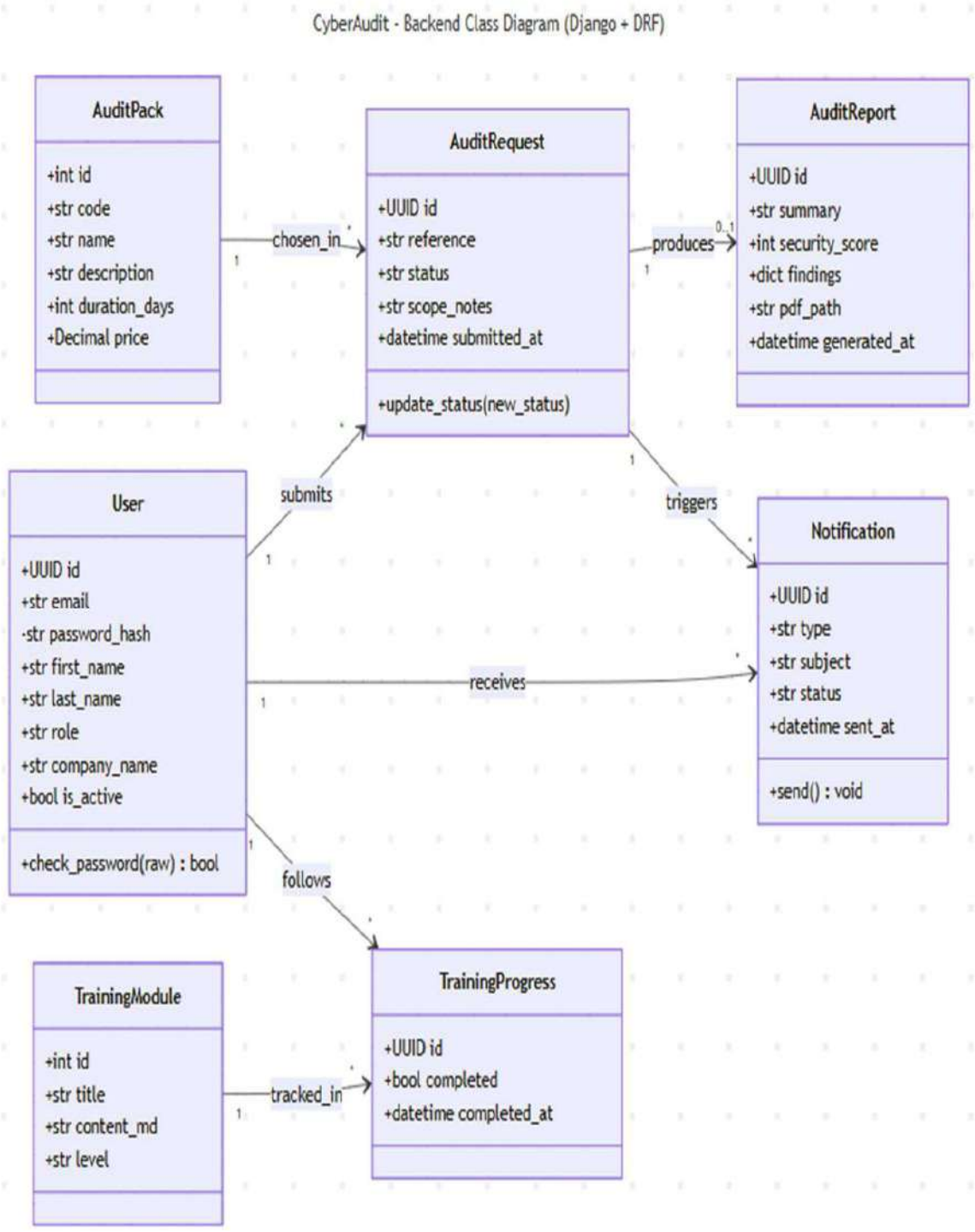


Diagramme de classes back-end (User, AuditPack, AuditRequest, AuditReport, Notification, TrainingModule, TrainingProgress).

Détail des attributs et méthodes des classes back-end

Classe	Attributs principaux	Méthodes
User	id (UUID, PK), email (unique), password_hash, first_name, last_name, role (client/admin), company_name, is_active, created_at	check_password(raw): bool
AuditPack	id, code (audit/sécurité/protection/premium), name, description, duration_days, price	
AuditRequest	id (UUID, PK), reference (ex. CYB-2026-0042), client_id (FK), pack_id (FK), status, scope_notes, submitted_at, updated_at, completed_at	update_status(new_status)
AuditReport	id (UUID, PK), audit_request_id (FK, unique), summary, security_score (0-100), findings (JSON), pdf_path, generated_at	build_pdf(), get_download_url()
Notification	id (UUID, PK), user_id (FK), request_id (FK, nullable), type (request_received/status_changed/report_ready), status (queued/sent/failed), sent_at	send(), retry()
TrainingModule / TrainingProgress	TrainingModule : id, title, content_md, level. TrainingProgress : id, user_id (FK), module_id (FK), completed, completed_at	

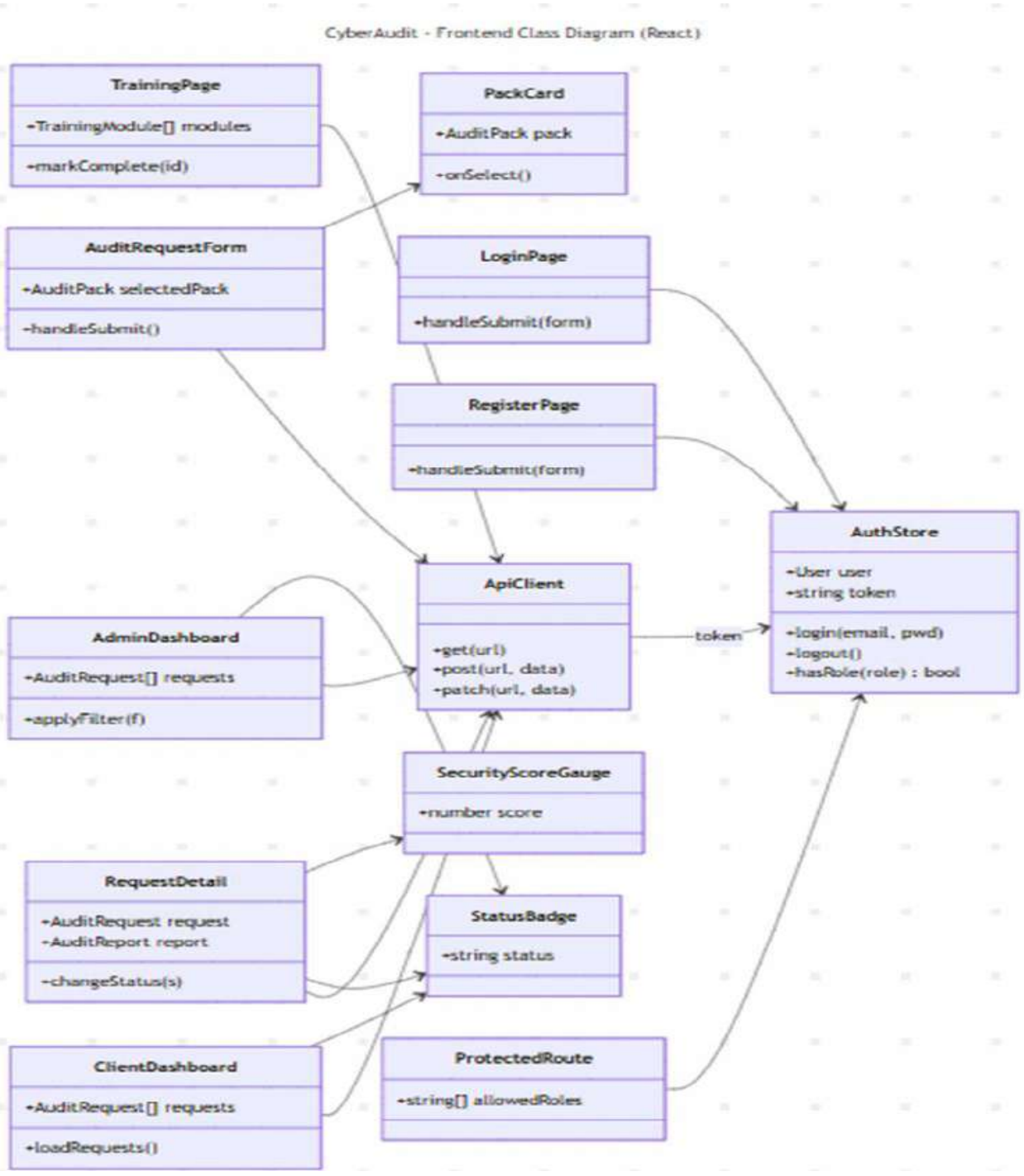


Diagramme de classes front-end « organisation des composants et services React autour du contexte d'authentification ».

ApiClient est la couche HTTP centrale : elle expose get(url), post(url, data) et patch(url, data), utilisées par le reste de l'application pour communiquer avec le back-end, et lit le jeton d'authentification depuis *AuthStore* pour authentifier automatiquement les requêtes. *AuthStore* conserve l'état de session (utilisateur courant, jeton JWT) et expose login (email, pwd), logout() et hasRole(role): bool. *LoginPage* et *RegisterPage* exposent chacune *handleSubmit(form)* et dépendent d'ApiClient et d'AuthStore. *ProtectedRoute* est un garde-route (propriété allowedRoles) qui interroge AuthStore.hasRole(...) pour appliquer le contrôle d'accès basé sur les rôles. Côté client : *ClientDashboard* (*AuditRequest[], loadRequests()), AuditRequestForm (selectedPack, handleSubmit()), RequestDetail (changeStatus(s)) et TrainingPage*

(modules, markComplete(id)) réutilisent *StatusBadge* et *SecurityScoreGauge*. Côté administrateur, *AdminDashboard* (*requests, applyFilter(f))* réutilise les mêmes composants partagés.

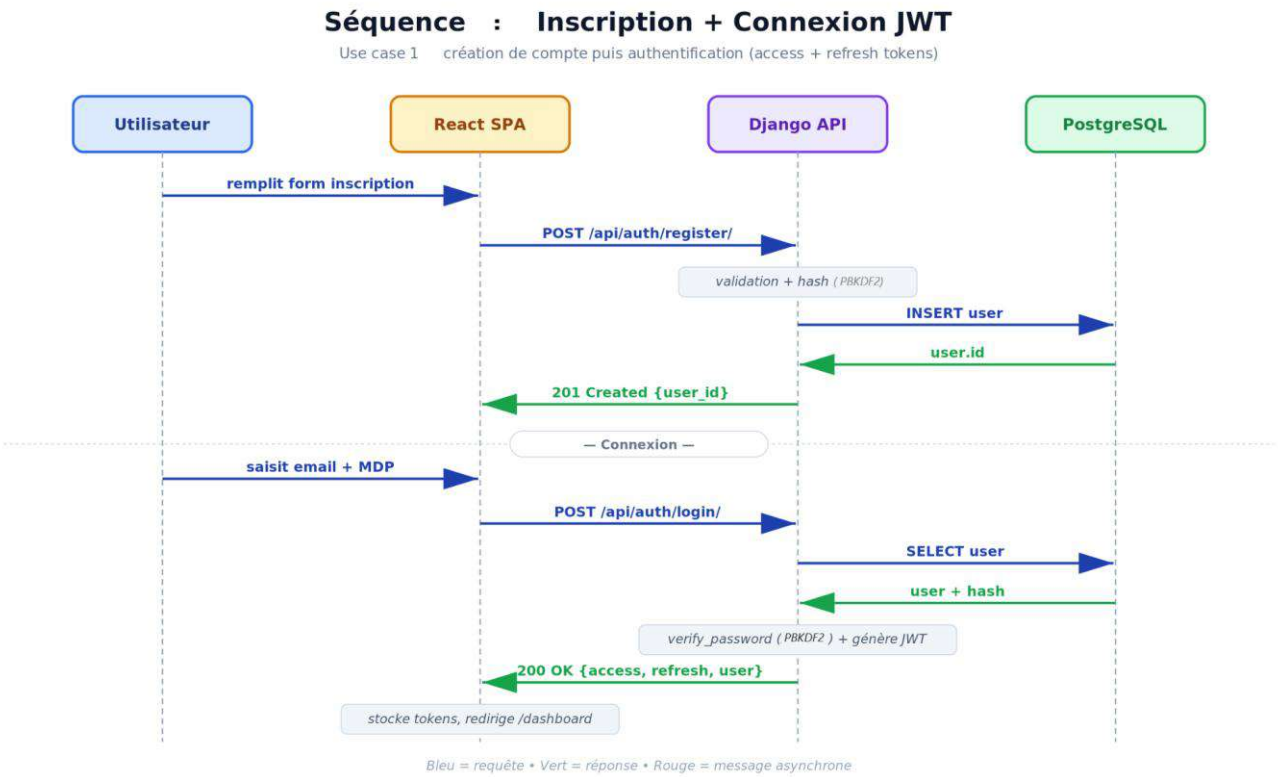
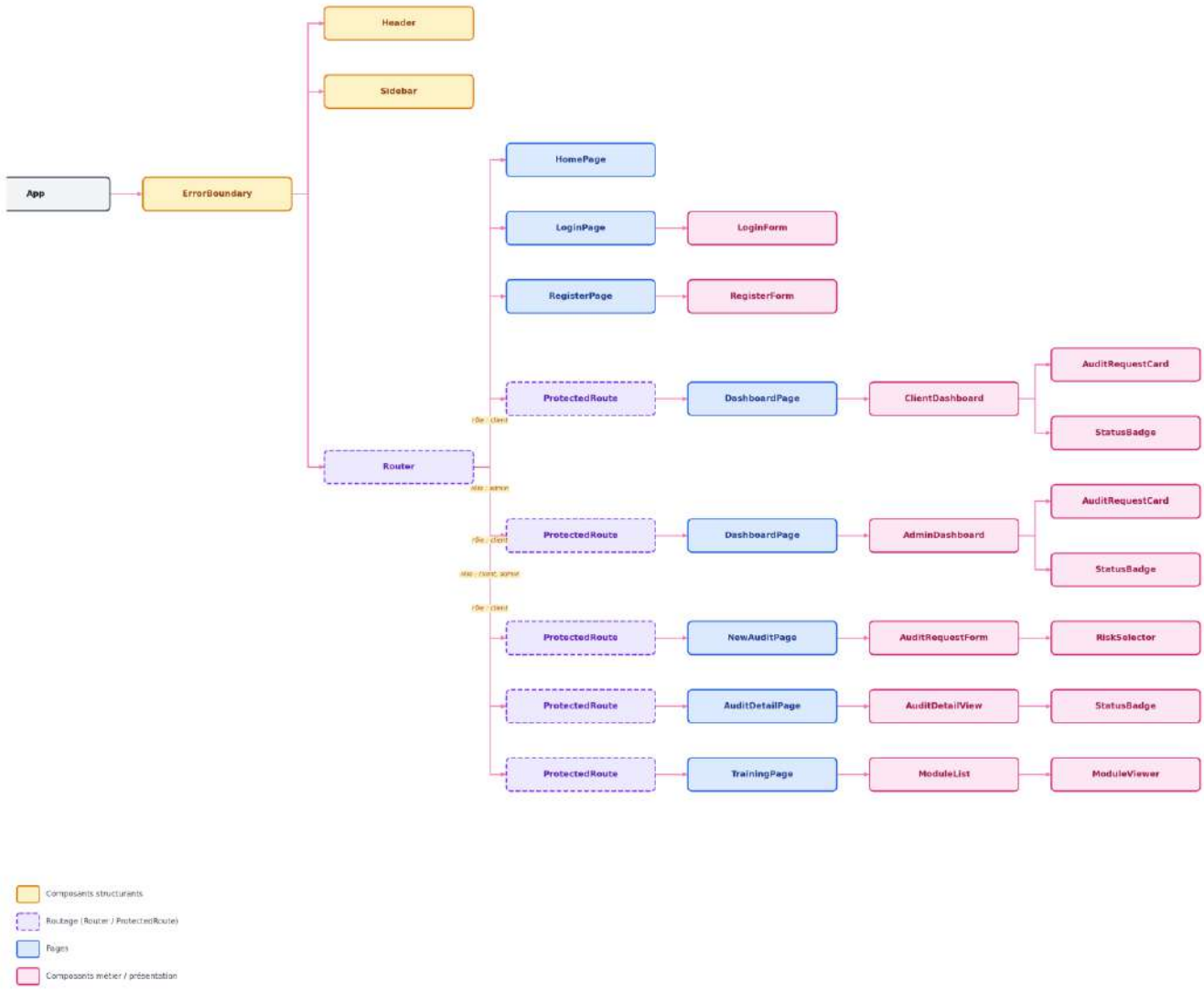


Diagramme de séquence « Inscription et connexion JWT ».

L'utilisateur saisit son courriel et son mot de passe ; le front appelle POST */api/auth/register/,* puis POST */api/auth/login/.* Django valide les identifiants, génère les jetons *access* et *refresh*, et les retourne au front, qui les stocke dans *AuthStore* avant de rediriger vers le tableau de bord correspondant au rôle.

Remarque : ce schéma, produit lors de la phase de conception amont, mentionne à titre d'illustration l'algorithme Argon2. L'implémentation finalement retenue pour le hachage des mots de passe est PBKDF2, paramétrage par défaut de Django (cf. section 8.4) ; seule la fonction de hachage diffère, le déroulé du flux d'inscription et de connexion représenté ci-dessus reste inchangé.



Arborescence des composants et navigation de CyberAudit & Solutions

Côté front-end, l'application s'organise autour d'un routeur protégé : chaque page sensible passe par ProtectedRoute, qui vérifie le rôle avant d'afficher le tableau de bord client ou administrateur. Tous les composants métier partagent une couche d'accès à l'API commune et des composants réutilisables comme le badge de statut ou la jauge de score de sécurité.

Annexe 6 : Ressources complémentaires du projet

Démonstration en ligne

https://portfolio-kappa-dun-65.vercel.app

Dépôt de code source

https://github.com/Tommy-JOUHANS/portfolio

Vidéo de démonstration

https://www.youtube.com/watch?v=jxrV9IZSC70

Annexe 7 : Top 10 des OSWAP



Le **Top 10 OWASP** présente les dix principales catégories de risques de sécurité auxquels les applications web sont exposées :

1. **Contrôle d'accès défaillant** : accès non autorisé à des données ou fonctionnalités.
2. **Défaillances cryptographiques** : protection insuffisante des données sensibles.
3. **Injection** : insertion de commandes malveillantes dans une application.
4. **Conception non sécurisée** : failles liées à une mauvaise conception de la sécurité.
5. **Mauvaises configurations de sécurité** : paramètres incorrects ou protections insuffisantes.
6. **Composants vulnérables ou obsolètes** : utilisation de bibliothèques et logiciels présentant des failles.
7. **Défaillances d'identification et d'authentification** : mécanismes de connexion insuffisants.
8. **Défaillances d'intégrité des logiciels et des données** : absence de garanties suffisantes sur l'intégrité des éléments utilisés.
9. **Journalisation et surveillance insuffisantes** : difficulté à détecter les attaques et les comportements suspects.
10. **Falsification de requêtes côté serveur** : exploitation du serveur pour accéder à des ressources internes non autorisées.

Lien avec le projet CyberAudit & Solutions

Le Top 10 OWASP constitue une référence importante pour identifier et prévenir les vulnérabilités des applications web. Dans le cadre de **CyberAudit & Solutions**, il permet de sensibiliser les utilisateurs aux risques informatiques et de guider les audits de sécurité afin de renforcer la protection des données et la fiabilité de la plateforme.

Objectif : anticiper les cyberattaques, réduire les vulnérabilités et développer des applications web plus sûres.